AOSP 进阶丛书 · 第二册 · ANDROID 13 (API 33)

系统启动主线逐函数精读

按下电源键到 Launcher 出现的每一步：两阶段 init 与 rc 触发瀑布、单线程事件驱动主循环、Zygote 预加载与 fork、SystemServer 三阶段服务总装与 boot phase 时间线——把开机变成白盒。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目录

前言：把开机变成你的白盒 2
第一章 启动全景：九个阶段 2
第二章 init 诞生：main() 的四个身份 3
第三章 第二阶段 init：SecondStageMain 逐段精读 6
第四章 init 语言：rc 文件的解析与求值 8
第五章 init 主循环：单线程事件驱动 15
第六章 Zygote：App 世界的祖先 17
第七章 SystemServer：三阶段服务总装 22
第八章 开机完成：systemReady 到 BOOT_COMPLETED 26
第九章 启动性能：主线上的优化战场 30
第十章 设计决策十问 30
第十一章 深水区：bootloader、fastboot 协议与 OTA 引擎 31
附录 A 观测实操手册 32
附录 B 关键文件清单（本册出场人物） 33
附录 C 自检题 33
附录 D 动手实验（一天量） 34
第十二章 属性触发引擎：init 事件机器的内部视角 34
第十三章 Zygote 孵化协议与 specialize：一次 fork 的完整解剖 39
第十四章 Watchdog：system_server 的心电图 44
第十五章 服务重启策略：init 的熔断器 47
教科书习题集（分级） 51
术语表（按拼音排序） 54
排障手册：启动问题定位 55

本册速览

九阶段总时序：BootROM→init 两阶段→Zygote→SystemServer→BOOT_COMPLETED
rc 触发瀑布与单线程事件循环 · Zygote 预加载与 fork 流水线
boot phase 100→1000 时间线 · sys.boot_completed 权威标志
深水区：bootloader 两级跳 · fastboot 协议 · GKI · update_engine
边读边做：dmesg/bootchart 对位九阶段，附录 D 五个实验

前言：把开机变成你的白盒

第一册给了你地图，本册带你沿主线行军：**按下电源键到 Launcher 出现**的每一步，都落到可验证的函数与行号上。这一条线串起了 init、rc 语言、属性系统、Zygote、ART、SystemServer、AMS——它们是 Android 系统开发的“文言文”，读懂它，后面所有子系统的阅读速度都会质变。

本册源码基线与第一册相同（AOSP Android 13，`$A` 代指源码树根）。建议配合真机或模拟器同步实验：`dmesg`、`logcat -b events` 里的每一行，本册都会告诉你它从哪个源文件打出。

第一章 启动全景：九个阶段

设计洞察 · 学习目标

1. 按顺序说出从 BootROM 到 BOOT_COMPLETED 的九个阶段；2. 指出三个分水岭（内核→用户态、native→Java、就绪→可用）及各自的标志事件；3. 知道 `sys.boot_completed` 是全机唯一的权威完成标志及其设置者；4. 建立"启动优化的战场在后三段"的时间量级直觉

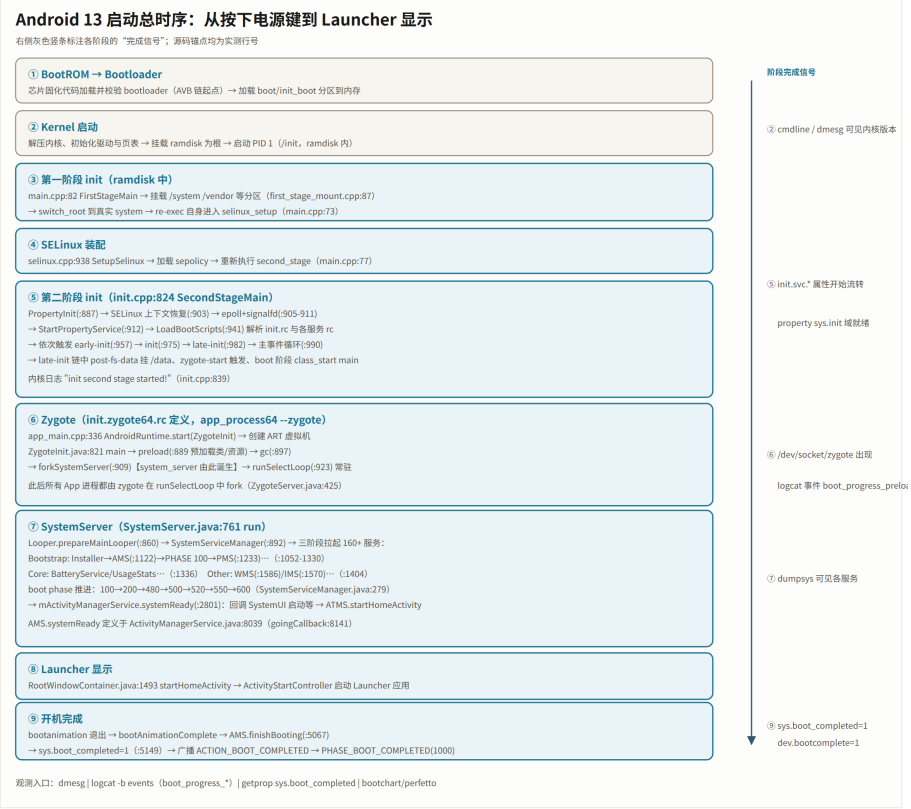


图 1 Android 13 启动总时序：BootROM 到 BOOT_COMPLETED 的九个阶段与各阶段完成信号

九个阶段里有三个分水岭值得先记住：

- **内核 → 用户态**：PID 1（init）诞生，此后的世界全部由 init 直接或间接孵化。
- **native → Java**：Zygote 创建 ART 虚拟机并预加载框架，此后 Framework 是 Java 的世界。

- **就绪 → 可用**：`AMS.systemReady()` 之后才允许启动 Home 与三方应用，`sys.boot_completed=1` 是全机“开机完成”的唯一权威标志（`ActivityManagerService.java:5149`）。

时间量级参考（中端机）：BootROM+bootloader 数十毫秒，内核一两秒，init 两阶段数百毫秒，zygote 预加载数百毫秒到一秒，SystemServer 两三秒。**启动优化的战场几乎全在后三段**——也就是本册第二到第九章覆盖的范围。

第二章 init 诞生：main() 的四个身份

> **设计洞察 · 学习目标**
> 1. 说出 init 二进制的四种身份及各自的触发条件（argv[0]/argv[1]）；2. 解释两阶段 init 与两次 re-exec 解决的"鸡生蛋"问题；3. 说明 ueventd 与 init 同二进制的空间与安全收益；4. 能跟踪一个 /dev 设备节点从内核 uevent 到 mknod 的完整链路

init 的可执行文件只有一个（`/system/bin/init`），但它以四种身份被反复执行。入口在 `system/core/init/main.cpp`：

cpp system/core/init/main.cpp:53-82（节选）

```
int main(int argc, char** argv) {
// 讲解注释①：优先级 -20（最高），保证早期初始化独占 CPU；
// 到主循环前恢复 0（:989）——"特权只在必要时持有"。
setpriority(PRIO_PROCESS, 0, -20);
// 讲解注释②：身份判据是 argv[0]——同一个二进制被 symlink 成 ueventd 时，
// 内核 exec 传来的 argv[0] 不同，init 借此"变身"（省一个 ELF 的空间）。
if (!strcmp(basename(argv[0]), "ueventd")) {
return ueventd_main(argc, argv); // 身份①：ueventd
}
if (argc > 1) {
if (!strcmp(argv[1], "subcontext")) { ... } // 身份②：SELinux 沙箱子进程
if (!strcmp(argv[1], "selinux_setup")) {
return SetupSelinux(argv); // 身份③：装载 SELinux
}
if (!strcmp(argv[1], "second_stage")) {
return SecondStageMain(argc, argv); // 身份④：第二阶段主体
}
}
// 讲解注释③：兜底身份=第一阶段 init。ramdisk 里的 /init 被内核拉起时
// 不带参数，走到这里；此后两次 re-exec 才依次切换到③④身份。
return FirstStageMain(argc, argv);
}
```

> **设计洞察 · 逐段精读：一个二进制的四次人生**
> ①argv[0]=ueventd（设备节点产房）；②subcontext（SELinux 沙箱里的 builtin 执行器）；③selinux_setup（装配策略后 re-exec）；④second_stage（真正的主角）。判断身份的手段全是**进程自己的属性**（argv/环境变量），不依赖任何外部状态——这是"最早期代码只能相信自己"的生存法则。

一码多身是精心设计：ramdisk 空间寸土寸金，一个二进制兼任 init/ueventd/subcontext 避免了多份 libc/ELF；身份切换只靠 argv——Unix 世界最古老的开关。

2.1 第一阶段：挂上真实的根

`FirstStageMain`（`first_stage_init.cpp:218`）只做一件事：**把散落的分区拼成一个可启动的根文件系统**。

- 从设备树读 fstab（`first_stage_mount.cpp:79` 的 `FirstStageMount`，`:87 DoFirstStageMount`），挂载 /system、/vendor 等；
- `PrepareSwitchRoot`（`:121`）与 `SwitchRoot("/first_stage_ramdisk")`（`:397`）：把根从 ramdisk 切到真实 system 分区；
- 检查 `/force_debuggable`（`:371`）支持解锁调试 ramdisk；
- 最后 **re-exec 自身**，参数 `selinux_setup`——因为装载 SELinux 策略需要“重启自身”来应用新的安全上下文。

2.2 SELinux 装配

`SetupSelinux`（`selinux.cpp:938`）加载 `/system/etc/selinux/precompiled_sepolicy` 等策略文件、恢复关键文件标签，然后再次 re-exec 进入 `second_stage`。至此系统完成了“先立规矩（安全策略）再办事”的前奏——**任何服务进程被拉起之前，它的 SELinux 域就已注定**。

2.3 并行伙伴：ueventd 与设备节点

`main.cpp:61` 的 argv[0] 分支让 init 的同一二进制兼任 ueventd（`system/core/init/ueventd.cpp`）。它在 early-init 前被拉起，监听内核 netlink 的 uevent，按 `ueventd.rc` 与 `ueventd.${ro.hardware}.rc` 规则创建 `/dev/*` 设备节点并设置权限——**所以 /dev 的世界里没有 udev，只有 ueventd**。主流程的 `wait_for_coldboot_done`（`init.cpp:960`）等它完成冷插扫描才继续，这解释了为什么两者的顺序是“ueventd 先跑、init 等、再触发”。firmware 加载（`firmware_handler.cpp`）也在 ueventd 侧：驱动请求固件时它从 /vendor/firmware 等路径搬运文件进内核。

> **设计洞察 · 为什么分两阶段**
> 第一阶段运行在 ramdisk 的临时根上，而 SELinux 策略、rc 剧本、服务二进制都在 /system 分区——不先挂分区就无事可做；而挂载动作本身又不该受旧 SELinux 状态污染。两阶段 + 两次 re-exec，用最朴素的方式解决了“鸡生蛋”：每个阶段只信任它已经亲手布置好的世界。

2.3.1 ueventd 深度走读：/dev 世界的缔造者

ueventd 与 init 是**同一个二进制**的不同身份（main.cpp:61 argv[0] 切换）。它在 early-init 之前被拉起，专职监听内核 uevent 并创建设备节点。

ueventd 的主循环

cpp system/core/init/ueventd.cpp（骨架+讲解注释）

```
int ueventd_main(int argc, char** argv) {
// 讲解注释①：解析 ueventd.rc 和 ueventd.${ro.hardware}.rc
// 规则格式：/dev/<name> <mode> <owner> <group>
// 例：/dev/graphics/* 0660 system graphics
// 讲解注释②：注册 netlink socket 监听内核 uevent
// 每个设备插拔/驱动注册 → 内核发 uevent → ueventd 收到
// 讲解注释③：冷启动扫描（coldboot）：
// 首次启动时主动扫描 /sys/class/* 下所有已注册设备
// → 为每个设备创建对应的 /dev 节点
// 讲解注释④：热插拔监听：
// USB 插入/拔出 → 内核发 uevent → ueventd 创建/删除节点
// → 触发 init 的 trigger（如 usb插入触发 vold 启动）
}
```

设备节点创建的完整链

text 从内核驱动到 /dev 节点

```
内核驱动加载（如 touch driver）
→ driver 注册 platform_device
→ 内核 kobject_uevent() 发 NETLINK_KOBJECT_UEVENT
→ ueventd 的 socket 收到消息
→ 解析 uevent（ACTION=add, DEVNAME=input/event3, SUBSYSTEM=input）
→ 查 ueventd.rc 匹配规则（权限/属主）
→ mknod("/dev/input/event3", mode, dev_t)
→ chmod + chown 按规则设置
→ 此后 App/framework 才能打开这个设备
// 讲解注释⑤：主流程 wait_for_coldboot_done（init.cpp:960）
// init 等 ueventd 完成冷启动扫描才继续触发后续 action——
// 因为 surfaceflinger 要打开 /dev/graphics/fb0，
// 没有设备节点=服务起不来
```

常见 ueventd 排障

症状	原因	排查
某设备节点不存在	ueventd 规则缺失或冷启动漏扫	cat /proc/bus/input/devices
节点权限错	ueventd.rc 规则错误	ls -la /dev/<name> 对比预期
USB 热插拔无反应	ueventd 死或 SELinux 拦截	logcat -s ueventd; getenforce

设计洞察 · 逐段精读：为什么不用 udev

Linux 桌面用 udev（一个功能更丰富的守护进程），Android 选了更简单的 ueventd——**启动阶段越少依赖越好**。ueventd 与 init 同二进制意味着零额外空间成本、共享 SELinux 策略、共享日志通道。这是"极简主义"的工程设计典范。

第三章 第二阶段 init：SecondStageMain 逐段精读

> **设计洞察 · 学习目标**
> 1. 按 :824-986 的源码顺序说出 SecondStageMain 的每个初始化步骤及其"潜台词"；2. 解释 signalfd、属性先行、epoll 中枢三个设计决策的动机；3. 说明 LoadBootScripts 的五级搜索路径与"分区边界即 rc 边界"；4. 记住 charger 模式分流与触发器入队顺序

SecondStageMain（init.cpp:824）是用户态世界的真正起点。以下按源码顺序分段精读（重要行号全部实测）。

3.1 环境自保（:824-886）

cpp system/core/init/init.cpp:824-886（节选）

```
int SecondStageMain(int argc, char** argv) {
if (REBOOT_BOOTLOADER_ON_PANIC) InstallRebootSignalHandlers();
boot_clock::time_point start_time = boot_clock::now();
SetStdioToDevNull(argv); // stdio 全部指向 /dev/null
InitKernelLogging(argv); // 日志改走 /dev/kmsg
LOG(INFO) << "init second stage started!";
...
struct sigaction action = {.sa_flags = SA_RESTART};
action.sa_handler = [](int) {}; // SIGPIPE 置为 no-op（:851-855）
sigaction(SIGPIPE, &action, nullptr);
...
WriteFile("/proc/1/oom_score_adj", ...); // init 永不被 LMK 杀（:858-863）
keyctl_get_keyring_ID(KEY_SPEC_SESSION_KEYRING, 1); // 会话密钥环（FBE 加密钥匙落点）:868
close(open("/dev/.booting", O_WRONLY | O_CREAT | O_CLOEXEC, 0000)); // “正在开机”哨兵 :871
```

这些琐碎动作各有一句“潜台词”：stdio 归零是因为子服务会继承 fd，泄漏一个 console fd 就等于开一个后门；SIGPIPE 不能杀 init，否则任何子进程退出都可能带走 PID 1；/dev/.booting 哨兵告知固件加载器“稍安勿躁”。**系统级代码的第一素养：每个“为什么”都有安全或时序答案。**

3.2 基础设施三件套（:887-930）

cpp system/core/init/init.cpp:887-941（节选）

```
PropertyInit(); // ① 属性区与默认值 :887
...
SelinuxSetupKernelLogging(); // ② SELinux 日志通道 :901
SelabelInitialize(); // 加载 file_contexts :902
SelinuxRestoreContext(); // 递归恢复全盘标签 :903
Epoll epoll;
epoll.Open(); // ③ 事件中枢 :905-908
InstallSignalFdHandler(&epoll); // SIGCHLD → signalfd（不嵌信号处理器）:910
InstallInitNotifier(&epoll);
StartPropertyService(&property_fd); // 属性写仲裁 socket :912
...
SetupMountNamespaces(); // 每个服务可获独立 mount ns :932
```

```
ActionManager& am = ActionManager::GetInstance();
ServiceList& sm = ServiceList::GetInstance();
LoadBootScripts(am, sm); // 解析全部 .rc :941
```

三个设计细节值得放大：

- **signalfd 而非 signal handler**：信号处理器会在任意线程任意字节码中间打断执行，是数据竞争之源；signalfd 把 SIGCHLD 变成 epoll 里的一个普通 fd，与 init 单线程模型严丝合缝（:829-832 的注释明言 signalfd 之前禁止起线程）。
- **属性先于一切服务**：后面所有 rc 的 `on property:` 触发器都依赖属性区存在。
- **LoadBootScripts 的搜索路径**（:307-322）：`/system/etc/init/hw/init.rc` → `/system/etc/init` → `/system_ext/etc/init` → `/vendor/etc/init` → `/odm/etc/init`——**分区边界即 rc 边界**，vendor 的服务只在 vendor 的 rc 里声明。

3.4 属性仲裁：handle_property_set_fd

cpp system/core/init/property_service.cpp:527-604（节选）

```
static void handle_property_set_fd() {
int s = accept4(property_set_fd, nullptr, nullptr, SOCK_CLOEXEC);
...
struct ucred cr; // 对端凭证：内核背书的 pid/uid
socklen_t cr_size = sizeof(cr);
getsockopt(s, SOL_SOCKET, SO_PEERCRED, &cr, &cr_size);
...
if (mac_len > 0) { // SELinux 上下文随请求携带
char* source_ctx = ...;
uint32_t result = HandlePropertySet(prop_name, prop_value,
source_ctx, cr, &socket, &error);
```

`SO_PEERCRED` 是 Unix 域套接字的“身份证读取器”——**凭证由内核填写，客户端无法伪造**。这与 Binder 驱动填充 `sender_euid`（第三册 `IPCThreadState.cpp:1270`）是同一个安全哲学的两处实现：身份必须在内核边界采集。

3.3 点火：触发器入队（:953-986）

cpp system/core/init/init.cpp:953-986（节选）

```
am.QueueBuiltinAction(SetupCgroupsAction, "SetupCgroups");
...
am.QueueEventTrigger("early-init"); // :957
am.QueueBuiltinAction(wait_for_coldboot_done_action, ...); // :960 等 ueventd 冷插
...
am.QueueEventTrigger("init"); // :975
std::string bootmode = GetProperty("ro.bootmode", "");
if (bootmode == "charger") {
am.QueueEventTrigger("charger"); // 充电模式：不开机只充电 :979
} else {
am.QueueEventTrigger("late-init"); // :982 正常开机主干
}
am.QueueBuiltinAction(queue_property_triggers_action, ...); // :986
```

注意 charger 分支：插着充电器开机时 init 走完全不同的剧本——不启动 zygote、不进 boot 阶段，只点亮充电界面。**同一个 init，两套人格，全靠触发器分流**。

第四章 init 语言：rc 文件的解析与求值

设计洞察 · 学习目标

1. 说出 late-init 瀑布的九级子触发器顺序及其文件系统依赖动机；2. 掌握 rc 的 service/on/import 三类语法与全部常用命令；3. 读懂 Parser→Service/Action 对象的解析期转换与"解析=建对象、class_start=fork"的语义分离；4. 能从 action_manager 求值日志反推 rc 的执行时序

4.1 剧本总谱：late-init 瀑布

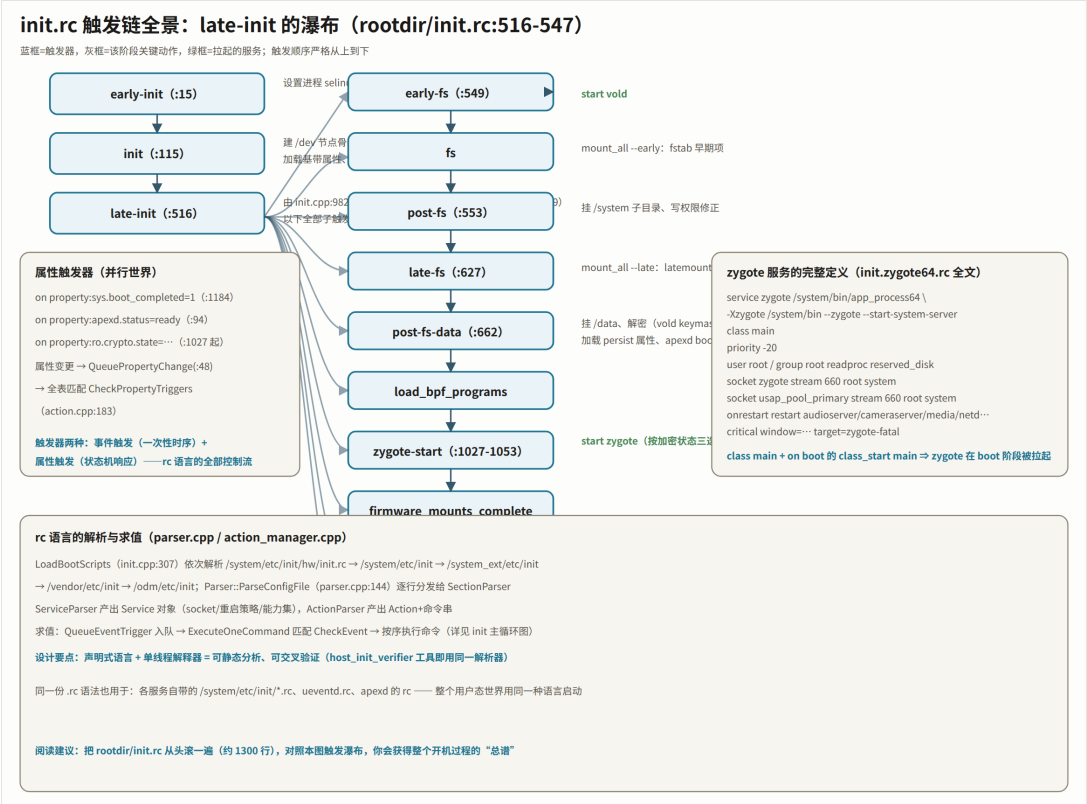


图 2 init.rc 触发链全景：late-init 的瀑布式子触发器、属性触发器并行世界与 zygote 服务完整定义

`rootdir/init.rc` 的 `on late-init`（:516-547）段是全剧总谱，子触发器严格按书写顺序入队：`early-fs → fs → post-fs → late-fs → post-fs-data → load_bpf_programs → zygote-start → firmware_mounts_complete → early-boot → boot`。每个名字都对应“文件系统与关键服务可用性”的一级台阶，`zygote-start` 之前必须完成 /data 解密（`post-fs-data`），因为应用数据与 dex 都在 /data 上。

zygote 的启动条件用三个属性触发器表达（:1027-1053）：`ro.crypto.state=unencrypted / unsupported / encrypted&&file`——**加密状态决定 zygote 何时可以跑**，这就是“顺序依赖”在声明

式语言里的表达方式。

4.2 解析器三件套

`Parser::ParseConfigFile`（`parser.cpp:144`）读文件后按第一个 token 分发：`service` 交给 `ServiceParser`（构造 `Service` 对象：二进制路径、class、socket、重启策略、SELinux 域、能力集），`on` 交给 `ActionParser`（构造 `Action` + 命令串），`import` 递归解析。同一套解析器还服务于 `host_init_verifier`（构建期校验 rc 的正确性）——**一套语法，三处使用（开机求值、静态校验、CTS 检查）**，这是声明式设计换来的红利。

4.3 Action 求值：一次一条

cpp system/core/init/action_manager.cpp:67-94（节选）

```
void ActionManager::ExecuteOneCommand() {
{
auto lock = std::lock_guard{event_queue_lock_};
while (current_executing_actions_.empty() && !event_queue_.empty()) {
for (const auto& action : actions_) {
if (std::visit([&action](const auto& event) {
return action->CheckEvent(event); }, event_queue_.front())) {
current_executing_actions_.emplace(action.get());
}
}
event_queue_.pop();
}
}
...
action->ExecuteOneCommand(current_command_); // 只执行队首 action 的第 N 条命令
++current_command_;
```

一次主循环只推进**一条命令**。这意味着一段 50 条命令的 action 会与其他触发器交错执行——没有谁能霸占 init。代价是吞吐，但 init 的 KPI 从来不是吞吐，是**确定性**。

4.4 服务孵化：do_class_start 与 Service::Start

`on boot` 里的 `class_start core/main` 走到 `builtins.cpp:163`：

cpp system/core/init/builtins.cpp:163-176（节选）

```
static Result<void> do_class_start(const BuiltinArguments& args) {
for (const auto& service : ServiceList::GetInstance()) {
if (service->classnames().count(args[1])) {
if (auto result = service->StartIfNotDisabled(); !result.ok()) { ... }
}
}
return {};
}
```

`Service::Start`（`service.cpp:524`）是所有守护进程的产房：先建 socket（`:613`，所以 zygote 的 `/dev/socket/zygote` 在 fork 前就存在），再 `fork()`（`:633`），子进程 `RunService` 后 `exec` 目标二

进制（:638）；父进程登记 pid、写 oom_score_adj（:647）、创建 cgroup（:663）、`NotifyStateChange("running")`（:684）——**把 `init.svc.zygote` 属性置为 running**。属性即状态机，`getprop init.svc.zygote` 因此成为运维常识。

4.5 rc 语言语法要素速查

语句	语义	示例	求值时机
service	声明服务及其属性	service zygote /system/bin/app_process64 …	解析期建对象，class_start 时启动
on <trigger>	声明动作段	on boot / on property:sys.boot_completed=1	触发器命中后逐命令执行
trigger <name>	排队子触发器	trigger zygote-start（写在 late-init 内）	作为命令入队
import	引入其他 rc	import /init.${ro.zygote}.rc	解析期展开
class_start 等 builtin	命令	class_start main / mount / setprop / exec	Action 求值时执行
onrestart / ondestruct	服务挂钩	onrestart restart audioserver	服务重启时执行
critical / oneshot / disabled	服务策略	critical window=… target=zygote-fatal	Reap 时决定重启行为

语言细节最能体现“声明式”的地方：`service` 段里**没有任何控制流**，属性就是断言；`on` 段的命令序列是纯顺序；一切分支由触发器与属性在运行期决定。这也是 rc 文件能被 `host_init_verifier` 在构建机上完整校验的原因——不存在“跑到才知道”的行为。

4.6 init.rc 完整语言参考（教科书级速查）

service 段全部属性

属性	语法	含义	示例
name + path	service X /path/to/bin	服务名+可执行路径	service zygote /system/bin/app_process64
class	class <name>	启动类别（class_start 批量启动）	class main
user / group	user <u> group <g>	运行身份	user root group root readproc
socket	socket <n> <t> <p> <o> <g>	创建 Unix socket	socket zygote stream 660 root system

属性	语法	含义	示例
oneshot	oneshot	只跑一次不重启	--
onrestart	onrestart <cmd>	重启时执行	onrestart restart audioserver
critical	critical	死则整机重启	critical window=... target=...
disabled	disabled	不随 class_start 启动	--
seclabel	seclabel <label>	SELinux 域标签	seclabel u:r:zygote:s0
priority	priority <n>	进程优先级	priority -20
task_profiles	task_profiles <p>	cgroup 任务档	task_profiles ProcessCapacityHigh
interface	interface <name>	HAL 服务声明	interface android.hardware.power.IPower/default
writepid	writepid <file>	写 pid 到文件（watchdog 用）	--

on 段全部命令（常用 20 个）

命令	语法	含义
mkdir	mkdir <path> [mode] [owner] [group]	创建目录
mount	mount <type> <dev> <dir> [flag...]	挂载文件系统
chmod / chown	chmod <mode> <path>	改权限/属主
setprop	setprop <name> <value>	设属性
start / stop / restart	start <svc>	控制服务
class_start	class_start <class>	批量启动
exec	exec [<seclabel>] [--] <cmd> <args>	执行外部命令（阻塞）
wait	wait <path> [timeout]	等待文件出现
copy	copy <src> <dst>	复制文件
write	write <path> <content>	写文件
symlink	symlink <target> <link>	创建符号链接
trigger	trigger <event>	排队子触发器
verity_update_state	verity_update_state <mount>	更新 dm-verity 状态

import 语句

text import 的解析顺序（LoadBootScripts :307）

```
/system/etc/init/hw/init.rc ← 总谱
/system/etc/init/*.rc ← 各服务自带 rc
/system_ext/etc/init/*.rc ← 扩展分区
/vendor/etc/init/*.rc ← vendor 分区
/odm/etc/init/*.rc ← odm 分区
// 讲解注释：分区边界即 rc 边界——vendor 的服务只在 vendor 的 rc 里声明
```

设计洞察 · 逐段精读：rc 语言的设计哲学

声明式（无控制流，属性即断言）+ **触发器**（事件驱动+属性驱动）+ **单一解析器**（开机求值/静态校验/CTS 检查三处复用）。这三个设计决策让 init.rc 保持了 15 年的稳定性——**好的 DSL 是"少即是多"的教科书。**

9.1 启动性能分析实战：用 bootchart 定位慢在哪

bootchart 是什么

bootchart 是 init 内建的启动时间线采集器（system/core/init/bootchart.cpp）：设置 `persist.init.bootchart` 后，init 记录各 action/service 的启动时间戳，配合 `grabbootchart.sh` 产出甘特图。

五步实操

bash bootchart 实操五步（可直接执行）

```
# 步骤 1：开启采集
adb shell setprop persist.init.bootchart 1
# 步骤 2：重启
adb reboot
# 步骤 3：等待开机完成
adb wait-for-device shell 'while [ "$(getprop sys.boot_completed)" != "1" ]; do sleep 1;
done'
# 步骤 4：停止并导出
adb shell setprop persist.init.bootchart stop
adb pull /data/bootchart /tmp/bootchart
# 步骤 5：生成甘特图
cd /tmp/bootchart && tar -czf bootchart.tgz *
python3 system/core/init/grabbootchart.py /tmp/bootchart
```

典型 bootchart 输出解读

text bootchart 甘特图（简化示意）

```
时间(ms) 0 500 1000 1500 2000 2500 3000 4000
init ████████████ ← init 本体
ueventd ██████ ← 设备节点
vold ████████ ← 卷管理
surfaceflinger ████████████████ ← 图形
zygote ████████████████████████ ← preload 最耗时
system_server ████████████████████████████████ ← 最多服务
bootanimation ████████████████████████████████← 动画直到 BOOT_COMPLETED
// 讲解注释①：zygote 段长的原因：preloadClasses（3000+ 类）
// → 优化方向：裁剪 preloaded-classes（Go 版走 --enable-lazy-preload）
// 讲解注释②：system_server 段长的原因：PMS 扫描全机 APK
```

```
// → 优化方向：并行扫描（SystemServerInitThreadPool）
// 讲解注释③：bootanimation 到 BOOT_COMPLETED 才停
// → 优化方向：提前放行动画（AMS.systemReady 不必等全部服务）
```

常见优化手段及效果

优化	预期收益	风险
裁剪 preloaded-classes	-500ms~1s	首帧变慢（类未预载）
lazy preload（Go 版默认）	-300ms~800ms	首个 App 启动变慢
PMS 并行扫描	-500ms~2s	稳定性（部分设备不稳定）
dex2oat 提前到 OTA 后	-1s~3s	OTA 后首次开机变慢
SystemServer 线程池并行初始化	-300ms~1s	需要梳理依赖
冻结 bootanimation 帧	省 0ms（体验优化）	--

设计洞察 · 逐段精读：性能优化的方法论

度量先行（bootchart 定位）→ **找瓶颈**（zygote preload？PMS 扫描？system_server？）→ **定向优化**（不要盲目调）→ **复验**（重跑 bootchart 对比）。这个"度量→分析→优化→验证"闭环是所有系统性能工作的通用方法论——比记住任何单个数字都值钱。

源码锚点 · 本章锚点速查

parser.cpp:144 ParseConfigFile / :238 ParseConfig 目录入口
action_manager.cpp:43 QueueEventTrigger / :57 QueueBuiltinAction / :67 ExecuteOneCommand
builtins.cpp:163 do_class_start；命令表 :1387 起
service.cpp:524 Start / :633 fork / :684 NotifyStateChange / :258 Reap
rootdir/init.rc:516-547 late-init 瀑布 / :1027 zygote-start 三条件

4.7 解析器核心走读：从文本到内存对象

init.rc 是纯文本文件——解析器把它变成内存中的 Service/Action 对象。这段代码是**声明式 DSL 解析的教科书**。

Parser 主流程

cpp system/core/init/parser.cpp（骨架+逐行注释）

```
bool Parser::ParseConfigFile(const std::string& path) {
// 讲解注释①：读取文件内容到内存
auto config_contents = ReadFile(path);
```

```
if (!config_contents.ok()) return false;
// 讲解注释②：逐行解析（ParseData → ParseLine）
// 每行按第一个 token 分发：
// "service" → ServiceParser（下一节）
// "on" → ActionParser
// "import" → 递归调用 ParseConfig
ParseData(path, &config_contents.value());
return true;
}
// ParseLine 的分发逻辑
void Parser::ParseLine(std::vector<std::string>&& args, const std::string& filename, int
line) {
if (args[0] == "service") {
// 讲解注释③：ServiceParser::ParseSection
// args[1]=服务名，args[2]=可执行路径
// 后续行（class/socket/user等）→ ParseLineSection
service_parser_->ParseSection(std::move(args), filename, line);
} else if (args[0] == "on") {
// 讲解注释④：ActionParser::ParseSection
// args[1..]=触发条件（trigger 或 property:name=value）
// 后续行（命令）→ ParseLineSection 追加到 Action
} else if (args[0] == "import") {
// 讲解注释⑤：递归解析（支持变量展开 ${ro.hardware}）
ParseConfig(args[1]);
}
}
```

Service 对象的完整生命周期

text rc → Service 对象 → fork → 运行 → 死亡 → 重启

```
rc 文件：Service 对象：OS 进程：
"service zygote ┌──────────────────┐
/system/bin/app_process64 │ name_: "zygote" │
--zygote │ pathname_: "..." │
class main" │ classnames_:{main}│
│ sockets_: [zygote]│
│ onrestart_: [...] │
│ pid_: 0（未启动）│
└─────────┬────────┘
│ class_start main
▼
┌──────────────────┐ fork+exec
│ Service::Start() │──────────────→ [zygote 进程]
│ socket 创建 │
│ pid_ = fork() │
│ SVC_RUNNING │
└─────────┬────────┘
│ SIGCHLD（进程退出）
▼
┌──────────────────┐
│ Service::Reap() │
│ 决定是否重启 │
│ critical→重启整机 │
│ oneshot→不重启 │
```

```
| 默认→延迟重启 |
└──────────────┘
```

一个 service 段被解析后的内存状态

text 解析 init.zygote64.rc 后的 Service 对象

```
Service {
name_: "zygote"
pathname_: "/system/bin/app_process64"
args_: ["-Xzygote", "/system/bin", "--zygote", "--start-system-server"]
classnames_: {"main"} // class_start main 触发
sockets_: [{name:"zygote", type:"stream", perm:660, uid:"root", gid:"system"}]
onrestart_: [
exec_background - system system -- /system/bin/vdc volume abort_fuse
write /sys/power/state on
restart audioserver
restart cameraserver
restart media
...
]
priority_: -20
seclabel_: "u:r:zygote:s0"
flags_: SVC_CRITICAL | SVC_RESTARTEXITING
pid_: (运行时填充)
}
```

设计洞察 · 逐段精读：解析器设计的三个精妙之处

①**同一解析器三处使用**：开机求值、构建期校验（host_init_verifier）、CTS 检查——声明式 DSL 的复用价值。②**延迟启动**：解析=建对象，class_start=真正 fork——语义与执行分离。③**变量展开在解析期**：${ro.hardware} 在 ParseConfig 时就展开（不是运行时）——保证确定性。

第五章 init 主循环：单线程事件驱动

设计洞察 · 学习目标

1. 背出主循环六步骨架（查关机→推一条命令→调度重启→epoll 等待→先收尸→处理 fd）；2. 解释"先 reap 再处理其他事件"防住的竞态；3. 说出 epoll 超时如何精确到服务重启节拍；4. 论证 init 选择单线程的三条理由与代价



图 3 init 主循环：epoll 事件源、ActionManager 事件队列、命令执行与服务生命周期，附单线程设计动机

主循环（`init.cpp:990-1056`）的骨架只有五步：查关机命令 → `am.ExecuteOneCommand()`（推进一条命令）→ `HandleProcessActions()`（服务重启调度，并算出下次唤醒时间）→ `epoll.Wait(timeout)` → 处理就绪 fd（**先 reap 子进程再处理其他事件**，`:1025-1028` 注释解释了原因：防止“服务已死但还没收尸，ctl.start 请求却被拒”的竞态）。

为什么单线程（图右下详述）：无锁可推理、逐命令公平、行为确定性可度量。对照第三册你会看到 system_server 的 Binder 线程池世界——两种模型各自服务“顺序的世界”与“并发的世界”。

5.1 主循环源码精读

cpp system/core/init/init.cpp:990-1031（节选）

```
while (true) {
auto epoll_timeout = std::optional<std::chrono::milliseconds>{kDiagnosticTimeout};
auto shutdown_command = shutdown_state.CheckShutdown();
if (shutdown_command) { HandlePowerctlMessage(*shutdown_command); } // 关机指令
if (!(prop_waiter_state.MightBeWaiting() || Service::is_exec_service_running())) {
am.ExecuteOneCommand(); // ① 每轮只推一条命令
}
if (!IsShuttingDown()) {
auto next_process_action_time = HandleProcessActions(); // ② 服务重启调度
if (next_process_action_time) {
epoll_timeout = ceil<milliseconds>(*next_process_action_time - now());
}
}
...
auto pending_functions = epoll.Wait(epoll_timeout); // ③ 等事件/定时
```

```
...
ReapAnyOutstandingChildren(); // ④ 先收尸（:1028）
for (const auto& function : *pending_functions) { (*function)(); } // ⑤ 再处理 fd
HandleControlMessages(); // ⑥ ctl.start/stop 请求
}
```

三个容易被忽略的精妙处：prop_waiter_state.MightBeWaiting()——属性等待（wait_for_prop 命令）期间暂停推进命令，保证测试脚本时序；HandleProcessActions 返回“下次需要醒来的时刻”，epoll 的超时因此**精确到服务重启节拍**，循环空转率为零；exec 服务（如 exec u:r:vendor_modem:s0 ...）执行期间主循环只等 SIGCHLD 并在超时后输出诊断（:1032-1050）——单线程模型对阻塞命令的补偿。

第六章 Zygote：App 世界的祖先

设计洞察 · 学习目标

1. 走通 app_process→AndroidRuntime→ZygoteInit.main 的五乐章；2. 列出 preload 清单并解释"多预载省首帧、少预载省内存"的权衡；3. 说清 forkSystemServer 出生证明各参数的含义；4. 用 COW 原理定量解释 Zygote 为全机节省的内存

6.1 出生：app_process 的使命



图 4 Zygote 的出生、预加载、forkSystemServer 与 runSelectLoop 孵化流水线，附 8 条设计动机

init 按 `init.zygote64.rc` 拉起 `/system/bin/app_process64 -Xzygote /system/bin --zygote --start-system-server`。`app_main.cpp` 解析参数后：

cpp frameworks/base/cmds/app_process/app_main.cpp:308-343（节选）

```
if (startSystemServer) {
args.add(String8("start-system-server")); // 传给 ZygoteInit
}
...
if (zygote) {
runtime.start("com.android.internal.os.ZygoteInit", args, zygote);
} else if (className) {
runtime.start("com.android.internal.os.RuntimeInit", args, zygote);
}
```

`AndroidRuntime::start`（同目录 `AndroidRuntime.cpp`）依次 `startVm`（创建 ART 虚拟机实例）→ `startReg`（注册数组、视图、Binder 等 JNI 方法）→ 反射调用 `ZygoteInit.main`——**native 世界把接力棒交给 Java 的准确坐标就是这一行反射调用**。

6.2 ZygoteInit.main：五大乐章

`ZygoteInit.java:821-938` 的结构极其工整：

java frameworks/base/core/java/com/android/internal/os/ZygoteInit.java:821-938（骨架）

```
public static void main(String[] argv) {
ZygoteHooks.startZygoteNoThreadCreation(); // :826 禁止起线程（fork 安全前提）
Os.setpgid(0, 0); // :830 独立进程组
...
if (!enableLazyPreload) {
preload(bootTimingsTraceLog); // :889 乐章①：预加载
}
gcAndFinalize(); // :897 乐章②：压实堆
Zygote.initNativeState(isPrimaryZygote); // :902 乐章③：native 状态（USAP 池 fd 等）
zygoteServer = new ZygoteServer(isPrimaryZygote); // :906 乐章④：创建 /dev/socket/zygote
if (startSystemServer) {
Runnable r = forkSystemServer(...); // :909 乐章⑤：fork system_server
if (r != null) { r.run(); return; } // 子进程直接跑 SystemServer.main
}
caller = zygoteServer.runSelectLoop(abiList);// :923 父进程：孵化循环（永不返回）
}
```

preload 的清单（`:134-173`）：`preloadClasses`（逐行实例化 `/system/etc/preloaded-classes` 里的三千多个类，源头是 `frameworks/base/preloaded-classes`）→ `cacheNonBootClasspathClassLoaders` → `preloadResources`（常用 drawable/字符串）→ `nativePreloadAppProcessHALs` → `maybePreloadGraphicsDriver` → 共享库/文字资源 → `WebViewFactory.prepareWebViewInZygote`。这份清单是 AOSP 用开机 trace 长期调优的产物——**多预载浪费内存，少预载首帧变慢**，它是一个全机级的权衡解。

6.3 前奏：preload 源码与清单哲学

java ZygoteInit.java:134-173（骨架）

```
static void preload(TimingsTraceLog bootTimingsTraceLog) {
beginPreload(); // ART 钩子：进入预加载态
bootTimingsTraceLog.traceBegin("PreloadClasses");
preloadClasses(); // ① 三千余类逐个 Class.forName
cacheNonBootClasspathClassLoaders();
preloadResources(); // ② drawable/字符串缓存
nativePreloadAppProcessHALs(); // ③ HAL 预热
maybePreloadGraphicsDriver(); // ④ 图形驱动预映射
preloadSharedLibraries(); // ⑤ System.loadLibrary 常驻
preloadTextResources();
WebViewFactory.prepareWebViewInZygote(); // ⑥ WebView 预备
endPreload();
warmUpJcaProviders(); // ⑦ 加密提供者预热
sPreloadComplete = true;
}
```

清单文件 `frameworks/base/preloaded-classes` 由 AOSP 的 `frameworks/base/tools/preload` 工具从真实开机 trace 挖掘而来（类加载计数超阈值者入选）。它是一份**活文档**：新版本会增删条目，厂商也会按机型裁剪。读它的正确姿势：随便挑一行类名，在 framework 源码里找到它，思考“为什么所有进程都需要它预载”——这份清单就是“Android 框架最内核工作集”的定义。

6.4 forkSystemServer：一个进程的出生证明

java frameworks/base/core/java/com/android/internal/os/ZygoteInit.java:711-789（节选）

```
/* 硬编码的出生证明 */
String[] args = {
// 讲解注释①：uid/gid 1000=system 用户；groups 是一串系统组
//  （1001=phone、3003=inet 等）——system_server 的"社交圈"在此写死。
"--setuid=1000", "--setgid=1000",
"--setgroups=1001,1002,1003,1004,1005,1006,1007,1008,1009,1010,1018,1021,"
+ "1023,1024,1032,1065,3001,3002,3003,3005,3006,3007,3009,3010,3011,3012",
// 讲解注释②：能力位图双份（permitted/effective）——CAP_KILL、CAP_NET_ADMIN
// 等 13 项（:685-699），出生即最小权限白名单。
"--capabilities=" + capabilities + "," + capabilities,
"--nice-name=system_server", // 讲解注释③：进程名（ps 可见）
"--runtime-args",
"com.android.server.SystemServer", // 讲解注释④：入口类（书2 第七章）
};
...
pid = Zygote.forkSystemServer(parsedArgs.mUid, parsedArgs.mGid, ...); // :770 native fork
...
if (pid == 0) { // :782 子进程侧
if (hasSecondZygote(abiList)) waitForSecondaryZygote(socketName);
zygoteServer.closeServerSocket(); // :787 关掉孵化 socket
return handleSystemServerProcess(parsedArgs); // :788 进入 SystemServer
}
```

这段“出生证明”回答了三个问题：system_server 的 uid/gid 为何是 1000（system 用户）；它为什么握有 CAP_KILL/CAP_NET_ADMIN 等 13 项能力而普通应用没有；它为什么必须由 zygote 长

fork 而不是 init 直接拉起（要继承预加载的整个 Java 世界）。

6.4 runSelectLoop：孵化流水线

ZygoteServer.java:425 起：Os.poll 监听服务 socket、已接会话 socket 与 USAP 池管道。新连接由 acceptCommandPeer（:541）接纳；会话可读则 processCommand（:552）解析孵化参数（uid/gid/ABI/capabilities/rlimits/seInfo…），调 Zygote.forkAndSpecialize（Zygote.java:347）。**子进程带 Runnable 返回**（:562）退出循环，经 zygoteInit（ZygoteInit.java:980）三步——RuntimeInit.commonInit、nativeZygoteInit（在 app_main.cpp:92 的 onZygoteInit 里 ProcessState::self()->startThreadPool()，**Binder 线程池在此刻诞生**，第三册详述）、applicationInit → ActivityThread.main。父进程继续守在 poll 上，等下一个孵化请求。

USAP 池（:438-513）：预孵化若干“未特化”进程填池，高峰期孵化只需“特化”免去 fork；池有补水水位线与延迟节流。这是 Android 10 引入的孵化优化，判断设备是否启用可看 init.zygote64.rc 的 socket usap_pool_primary 声明。

6.5 runSelectLoop 源码精读

java frameworks/base/core/java/com/android/internal/os/ZygoteServer.java:539-562（节选）

```
if (pollIndex == 0) {
// 讲解注释①：index 0 永远是服务 socket 本身——accept 新会话
//  （客户端是 system_server 里的 ZygoteProcess）。
ZygoteConnection newPeer = acceptCommandPeer(abiList);
peers.add(newPeer);
socketFDs.add(newPeer.getFileDescriptor());
} else if (pollIndex < usapPoolEventFDIndex) {
// 讲解注释②：已接会话可读=对方写来了一次孵化请求参数串。
ZygoteConnection connection = peers.get(pollIndex);
final Runnable command = connection.processCommand(this, multipleForksOK);
// 讲解注释③：fork 之后的分水岭——mIsForkChild 为真说明"我现在是孩子"，
// 带 command 退出循环（父进程永远留在这里）。
if (mIsForkChild) {
if (command == null) throw new IllegalStateException("command == null");
return command;
}
}
```

设计洞察 · 逐段精读：一次 fork 的"分身时刻"

③是全书最微妙的一行：processCommand 内部调用了 forkAndSpecialize——从这一刻起**世界上有两个线程执行流**，父子各自从同一行代码继续。孩子 return command（去当 App），父亲继续 for 循环（继续当产房）。判断自己是谁的唯一依据是 mIsForkChild 标志。理解"fork 之后代码被两人各走一遍"，你就理解了 Unix 进程模型的精髓。

processCommand 内部：读出参数字符串数组（--setuid=10023 --runtime-args ... android.app.ActivityThread）→ ZygoteArguments 解析 → forkAndSpecialize（Zygote.java:347）。**native 层 fork 后的差异处理**（com_android_internal_os_Zygote.cpp）是本册最深的一段 C++：子

进程要降 uid/gid、设置能力、关掉 zygote 的 socket 与不需要的 fd、挂上自己的 seinfo 标签——一个新沙盒的所有边条在两次系统调用间完成。

USAP 池的节奏（:438-513）：池空阈值触发 mUsapPoolRefillTriggerTimestamp 记时，延迟窗口过后批量 fork 补水（DELAYED 状态）；请求数高于池容时直接退化为普通 fork。它解决的是**冷启动风暴**：桌面上十几个 App 同时被点开时，fork 本身的耗时也会排队。

6.6 Zygote 的内存世界：为什么 fork 能省百 MB

预加载后的内存布局

text zygote 进程的地址空间（fork 前）

```
┌──────────────────────────────────────┐ 高地址
│ 栈（Stack） │
├──────────────────────────────────────┤
│ │
│ ↑ 空闲区域 ↑ │
│ │
├──────────────────────────────────────┤
│ 共享的只读映射（COW 的核心！） │
│ ┌──────────────────────────────┐ │
│ │ framework.jar (boot.art) │ │ ← 所有 App 共享的类+机器码
│ │ framework-res.apk 资源 │ │ ← 图标/字符串/布局
│ │ preloadClasses (3000+ 类) │ │
│ │ preloadResources (drawable) │ │
│ │ preloadSharedLibraries │ │
│ └──────────────────────────────┘ │
├──────────────────────────────────────┤
│ 堆（Heap，GC 管理） │ ← 每次 fork 的 COW 基线
├──────────────────────────────────────┤
│ .data / .bss │
├──────────────────────────────────────┤
│ .text（ART 自身代码） │
├──────────────────────────────────────┤ 低地址
// 讲解注释①："共享的只读映射"是 Zygote 的核心价值——
// fork 后子进程与父进程共享同一物理页（内核 PTE 标只读）
// 只有写入时才触发 page fault → 内核复制该页（Copy-On-Write）
// → 未写入的页永远共享 = 零内存成本
```

fork 后的 COW 演示

text fork 后：共享 vs 私有

```
父进程（zygote） 子进程（App）
┌──────────────┐ ┌──────────────┐
│ framework 类 │ ═══════ │ framework 类 │ ← 共享（只读）
│ "Button" 类 │ ═══════ │ "Button" 类 │ ← 共享
│ 堆：对象A │ ✗ │ 堆：对象A' │ ← COW（App 改了 A）
│ 堆：对象B │ ═══════ │ 堆：对象B │ ← 共享（没改）
└──────────────┘ └──────────────┘

// 讲解注释②：App 只为它改过的对象付内存——
// 典型 App 只修改 ~10% 的堆对象
// 所以每个 App 的实际增量内存 ≈ 总堆 × 10%
```

```
// 而不是从零分配整个堆
```

定量估算

项	无 Zygote（每进程独立加载）	有 Zygote（COW 共享）
framework 类	约 120 MB × N 进程	120 MB（一份物理页）
资源（drawable/字符串）	约 30 MB × N	30 MB（一份）
ART 运行时	约 10 MB × N	10 MB
App 自身堆	--	~5-20 MB（因 App 而异）
总计（10 个 App）	**~1.6 GB**	**~200-350 MB**

设计洞察 · 逐段精读：这就是为什么 Android 能跑 100+ 进程

没有 Zygote+COW，每个 App 进程独立加载 160 MB 框架世界——8GB 设备最多跑 ~40 个进程。有了 COW，100 个 App 共享同一份物理页，增量只有各 App 自己修改的部分。**Zygote 是 Android 多进程模型的"地基桩"**——没有它，"每个 App 独立进程"的安全模型在内存上就不可行。

第七章 SystemServer：三阶段服务总装

设计洞察 · 学习目标

1. 说出 run() 前奏的九步自我装配及其动机；2. 按依赖因果链复述 Bootstrap 批 15 项的顺序（为什么 Watchdog 第一、Installer 先于 AMS、PMS 后熟）；3. 解释三批划分与 boot phase 100→1000 的"全局进度条"机制；4. 会用 SystemServiceManager 的反射调用模型理解服务生命周期

7.0 run() 前奏代码精读

java frameworks/base/services/java/com/android/server/SystemServer.java:802-899（骨架）

```
Binder.setWarnOnBlocking(true); // :802 禁同步外呼（防优先级反转）
...
BinderInternal.setMaxThreads(sMaxBinderThreads); // :854 31 线程（:435 定义）
...
Looper.prepareMainLooper(); // :860 本线程成为系统主线程
System.loadLibrary("android_servers"); // :867 加载 native 服务实现
createSystemContext(); // :882 系统 ActivityThread 与 Context
mSystemServiceManager = new SystemServiceManager(mSystemContext); // :892
LocalServices.addService(SystemServiceManager.class, mSystemServiceManager); // :897
SystemServerInitThreadPool tp = SystemServerInitThreadPool.start(); // :899 并行初始化池
```

`createSystemContext` 值得单独说：system_server 里跑着“世界上第一个 ActivityThread”，它没有界面，但拥有完整的 Context 体系——此后所有系统服务的 `mContext` 都从这里来。理解“system_server 也是一个（特殊的）应用进程”，很多 API 行为（如 registerReceiver、ContentResolver）才讲得通。

PMS 为什么在 Bootstrap 批：AMS 启动即需查询“哪些是 persistent 应用要提前拉起、权限如何”；WMS/Home 更依赖包扫描结果。所以顺序是 Installer→AMS（先占位）→phase 100→PMS（扫包）→……→PMS.systemReady(:2712)→AMS.systemReady(:2801)——**AMS 先生、PMS 后熟**，中间隔着整个 Other 批。

7.1 run() 的前奏（:761-929）

`SystemServer.java:761` 的 `run()` 在进入服务启动前完成自我装配：记录启动计数属性（:767-769）、时区与 locale 校正（:777-799）、`Binder.setWarnOnBlocking(true)`（:802，**禁止 system_server 同步调用其他进程**——防止低优先级进程反向卡死系统）、`BinderInternal.setMaxThreads(31)`（:854，Binder 线程预算翻倍）、`Looper.prepareMainLooper()`（:860）、`System.loadLibrary("android_servers")`（:867）、`createSystemContext`（:882）、new `SystemServiceManager`（:892）并放进 `LocalServices`（:897，同进程服务总线）、`SystemServerInitThreadPool.start()`（:899，并行化非关键初始化）。

7.2 三阶段与依赖即顺序



图 5 SystemServer 三阶段服务启动清单与 boot phase 100→1000 全局时间线，附 run() 硬核细节

Bootstrap 批（:1052-1330）的原则是“后序服务的地基”：Installer（installd 客户端）→ **AMS**（:1122）→ boot phase 100（等默认显示屏，:1202）→ **PMS**（:1233，扫描全机 APK）→ UserManager → OtaDexopt。**Core 批**（:1336）是轻量统计类：BatteryService、UsageStats、WebViewUpdate…。**Other 批**（:1404-3059，224 个 trace 段）是大合唱：`InputManagerService`（:

1570）、**WMS**（:1586）、phase 480/500/520（:2665/2669/2756）、PMS.systemReady（:2712）、AMS.systemReady（:2801）、startSystemUi（:3053）。

SystemServiceManager.startService（SystemServiceManager.java:207）用反射调 new Service(Context) 构造服务，再调其 onStart()；startBootPhase（:279）向所有已启动服务广播 phase，服务在 onBootPhase(phase) 里做阶段挂钩（如 AMS 等到 550 才放行三方调用）。

7.3 Bootstrap 批完整清单（traceBegin 顺序）

顺序	服务	一句话职责
1	Watchdog(:1058)	死锁看门狗（先行注册才能监视后序）
2	SystemConfig	读取 system/etc/sysconfig 全局配置
3	PlatformCompat	平台兼容性框架（AMS/PMS 都依赖）
4	FileIntegrityService	fs-verity 文件完整性
5	**Installer**	installd 客户端（dexopt/安装执行）
6	DeviceIdentifiersPolicyService	设备标识合规
7	UriGrantsManagerService	URI 授权
8	PowerStatsService / IStats / Memtrack	能耗与统计
9	**ATM+AMS**(:1122)	调度中枢占位（setInstaller 关联：1124+）
10	DataLoaderManager / IncrementalService	增量安装支撑
11	PowerManagerService(:1147)	电源策略
12	ThermalManagerService / HintManager	温控与性能暗示
—	boot phase 100(:1202)	等待默认显示屏
13	**PackageManagerService**(:1233)	全机 APK 扫描与权限基线
14	UserManagerService	多用户
15	OtaDexoptService(:1266)	OTA 后的 dex 整理（后台化）

这张表是“依赖即顺序”的具象化：每一行都在为后面某行扫清障碍。

7.4 服务启动顺序的因果链详解

这张表解释了"为什么必须是这个顺序"——每行都是一条因果链。

Bootstrap 批内部的严格序列

text Bootstrap 批的 12 步依赖链（不可调换）

```
1. Watchdog
↑ 为什么第一：后序服务要注册到 Watchdog 监控死锁
2. PlatformCompat
↑ 为什么第二：AMS/PMS 都要查兼容性行为
3. Installer
↑ 为什么第三：AMS 安装系统包需要 dexopt → installd 客户端
4. ATM + AMS (:1122)
↑ 为什么第四：后序服务的组件调度都靠 AMS
| setInstaller 关联第 3 步的 Installer
5. boot phase 100（等默认显示屏）
↑ 为什么在此：AMS 要知道主屏尺寸才能调度 Activity
6. PMS (:1233)
↑ 为什么第五：AMS 要查"哪些 persistent 应用要提前拉起"
| WMS/Home 更依赖包扫描结果（Launcher 是 APK！）
7. UserManagerService
↑ 为什么第六：多用户元数据
8. OtaDexoptService
↑ 为什么最后：OTA 后的 dex 整理是后台任务
// 核心洞察：顺序不是随意的，是"依赖拓扑排序"——
// 后序服务构造器里直接调先序服务的接口，乱序=NullPointerException
```

为什么 Core 批不和 Bootstrap 合并

Core 批的服务（BatteryService、UsageStatsService 等）**没有重依赖**（不需要等 Installer/AMS/PMS），但也不是"无足轻重"——把它们的启动与 Bootstrap 并行执行（SystemServerInitThreadPool），可以缩短 wall time。**拆分的本质是并行化机会的边界**。

startOtherServices 的 224 段为何最长

Other 批从 `:1404` 到 `:3059` 跨越 1600+ 行——因为 200+ 服务各自有初始化条件、相互依赖、boot phase 挂钩、异常处理。**这是 SystemServer.java 最大的函数**，也是系统开发者的"日常阅读主场"。

> **设计洞察 · 逐段精读：面试终极串联题**
>
> "说出从按下电源键到 system_server 就绪的完整链"——这个答案就是本册全部内容。但加分项是**因果**："为什么 Installer 在 AMS 之前？因为 AMS 启动就要装系统包，没有 dexopt 客户端寸步难行。"——能说出因果链的候选人才真正理解了系统。

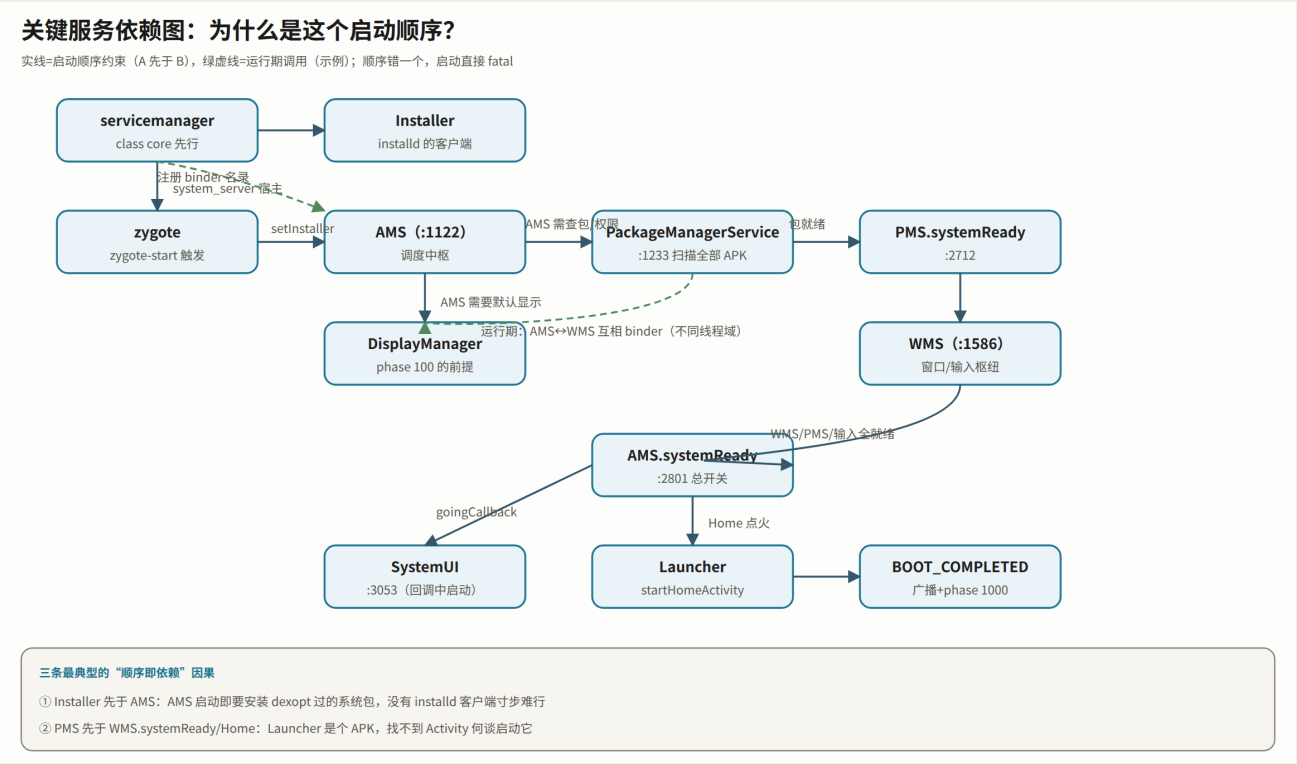


图 6 关键服务依赖图：servicemanager/zygote/Installer/AMS/PMS/WMS 的启动顺序因果与运行期调用

第八章 开机完成：systemReady 到 BOOT_COMPLETED

设计洞察 · 学习目标

1. 拆解 systemReady 两段式（内部就绪→goingCallback）与各 boot phase 的推进；2. 说清 Home 点火链与"Home 是第一个应用"；3. 走通 finishBooting 的动画门闩→sys.boot_completed=1→BOOT_COMPLETED 广播→phase 1000 全链条；4. 区分 sys.boot_completed 与 dev.bootcomplete 的语义与先后

8.0 systemReady 内部一览

java ActivityManagerService.java:8120-8144（节选）

```
t.traceBegin("retrieveSettings");
retrieveSettings(); // 回读全局设置
...
if (goingCallback != null) goingCallback.run(); // :8141 SystemServer 的回调在此执行
final int currentUserId = mUserController.getCurrentUserId();
...
t.traceBegin("ActivityManagerStartApps");
mBatteryStatsService.onSystemReady(); // 统计就位
...（后续：启动 persistent 应用、phase 550/600）
```

回调里 SystemServer 做的事（SystemServer.java:2801-3061）：通知 WindowManager“系统正在就绪”、启动 SystemUI（:3053）、开启 watchdog 正式监控、各服务 systemReady 链式传播。**两段式设计的动机**：AMS 的就绪依赖大量外部状态（包扫描、用户解锁、显示就绪），回调让“通知方”与“被通知方”解耦——AMS 不需要知道谁需要被叫醒。

8.1 systemReady 的两段式

ActivityManagerService.systemReady（ActivityManagerService.java:8039）先做内部就绪（清理临时进程、准备 user），goingCallback.run()（:8141）执行 SystemServer 传来的回调（SystemServer.java:2801-3061：启动 SystemUI、通知各服务收尾），然后按序 phase 550/600。Launcher 的点火链：ATMS.startHomeActivity（ActivityTaskManagerService.java:5874）→ RootWindowContainer.java:1493 → ActivityStartController.startHomeActivity——**Home 是第一个被启动的“应用”**。

8.2 finishBooting：动画是最后一道闸门

java frameworks/base/services/core/java/com/android/server/am/ActivityManagerService.java:5067-5086（节选）

```
final void finishBooting() {
synchronized (this) {
if (!mBootAnimationComplete) { // :5075 动画未完，记账后先返回
mCallFinishBooting = true;
return;
}
}
ZYGOTE_PROCESS.bootCompleted(); // :5085 通知 zygote（USAP 统计等）
VMRuntime.bootCompleted(); // :5086 通知 ART 进入“开机后”策略
...
SystemProperties.set("sys.boot_completed", "1"); // :5149 全机权威标志
```

开机动画退出 → bootAnimationComplete（:5240-5248）置位并补调 finishBooting()——**动画是用户可感知的最后一道闸门**，代码用 mBootAnimationComplete 门闩保证“广播必在动画之后”。随后 ACTION_BOOT_COMPLETED 有序广播发出、phase 1000 推进，接收方从三方 App 到 telephony 依次被唤醒。

8.3 BOOT_COMPLETED 的完整时序（教科书级逐步分解）

BOOT_COMPLETED 不是一条广播——它是一串事件的**终点**。以下是完整时序（含每步的系统属性变化）。

阶段 A：systemReady 之前（AMS 还没准备好）

text

```
t0: SystemServer startOtherServices 执行到 :2800 附近
|
├→ 所有 Bootstrap/Core/大部分 Other 服务已启动
├→ PMS.systemReady(:2712) 已调用
| └→ 包扫描完成，所有 APK 可查询
```

```
│
└→ mActivityManagerService.systemReady(goingCallback) :2801
│
├→ goingCallback.run() :8141
│ └→ SystemServer 的回调：
│ ├→ 恢复设置（retrieveSettings）
│ ├→ 通知各服务"系统即将就绪"
│ └→ 启动 SystemUI（startSystemUi :3053）
│
└→ 继续后续启动流程...
```

阶段 B：Home 点火

```text
t1: AMS 启动 Launcher
│
├→ ATMS.startHomeActivity (ActivityTaskManagerService.java:5874)
│ └→ RootWindowContainer.java:1499
│ └→ ActivityStartController.startHomeActivity
│ └→ "Home 是第一个被启动的应用"（书4 §一）
│
├→ Launcher 进程不存在 → ProcessList.startProcessLocked
│ └→ zygote fork → ActivityThread.main → onCreate
│
└→ Launcher 显示桌面 → 用户看到"桌面"
```

阶段 C：动画退出与最终标志

```text
t2: bootanimation 退出
│
├→ bootanimation 是独立进程（class core animation）
├→ 退出条件：service list 中出现 "surfaceflinger" 且 sys.boot_completed
│ 实际机制：PropertyOverlay 在 SF 侧监控 bootanim 状态
│
└→ bootAnimationComplete 回调到 AMS (:5240-5248)
└→ mBootAnimationComplete = true
└→ 若 mCallFinishBooting = true → 补调 finishBooting()
```

```text
t3: finishBooting (:5067)
│
├→ 动画门闩检查 (:5075-5083)
│ if (!mBootAnimationComplete) { mCallFinishBooting = true; return; }
│ // 动画没完就先记账等回调——"动画是最后一道闸门"
│
├→ ZYGOTE_PROCESS.bootCompleted() (:5085)
│ └→ 通知 zygote：开机完成（USAP 统计等）
│
├→ VMRuntime.bootCompleted() (:5086)
│ └→ 通知 ART：切换到"开机后"GC 策略
│
├→ SystemProperties.set("sys.boot_completed", "1") (:5149)
│ └→ ★ 全机权威标志！★
```

```
| └→ init 检测到此属性 → 触发 on property:sys.boot_completed=1 的 action
| └→ 通知各服务做开机后调整
|
└→ 发送 ACTION_BOOT_COMPLETED 有序广播
└→ 接收者按优先级依次收到（RECEIVE_BOOT_COMPLETED 权限）
└→ 最后 → SystemService.PHASE_BOOT_COMPLETED (1000)
```

阶段 D：用户可感知的"开机完成"

```text
t4：用户感知
|
├→ 桌面已显示（t1 的结果）
├→ 动画已退出（t2 的结果）
├→ BOOT_COMPLETED 广播已送达（t3 的结果）
|
└→ "开机完成" = 三个事件全部到位
时间上：桌面显示 < 动画退出 < BOOT_COMPLETED
（桌面可能比 BOOT_COMPLETED 早数秒）
```

关键属性的时间线

属性	设置时机	含义	谁消费
sys.boot_completed=1	finishBooting :5149	框架就绪	init 触发 on property
dev.bootcomplete=1	finishBooting 尾部	设备级完成	telephony 等
init.svc.zygote=running	Service::Start :684	服务状态机	运维工具
ro.runtime-init	SystemServer run() :767	启动计数	性能统计

设计洞察 · 逐段精读：为什么 BOOT_COMPLETED 由 AMS 而非 init 设置

"系统可用"的判据是**框架就绪**（Home 可启动、广播可送达）而不是"服务都在跑"——只有 AMS 有这个全局视角。init 只知道"服务起了"，不知道"服务能不能工作"。这就是为什么 sys.boot_completed 的设置者在 AMS.java:5149 而不是 init.cpp。

源码锚点 · 第八章锚点速查

SystemServer.java:2801 mActivityManagerService.systemReady(回调)
ActivityManagerService.java:8039 systemReady 定义 / :8141 goingCallback.run
ActivityTaskManagerService.java:5874 startHomeActivity / RootWindowContainer.java:1493 点火
ActivityManagerService.java:5067 finishBooting / :5075 动画门闩 / :5149 sys.boot_completed=1

第九章 启动性能：主线上的优化战场

> **设计洞察 · 学习目标**
> 1. 建立度量先行的优化方法论（bootchart→boot_progress→perfetto）；2. 说出预加载裁剪、并行化、USAP 孵化加速三个经典优化位的原理与风险；3. 理解"首次成本与稳态成本"的交易模型；4. 会用 boot_progress_* 事件给一次真实开机分段计时

读完白盒链路，性能话题就有了坐标。启动优化在 AOSP 里是持续十几年的工程，本册给出现代方法的骨架：

度量为先。`bootchart`（init 内建）画甘特图定位“哪个服务占了哪段时间”；`logcat -b events` 的 `boot_progress_*` 事件（preload_start/end、system_run 等）给出阶段锚点；perfetto（`external/perfetto`）可叠加 CPU/IO/调度视角。**没有度量的启动优化都是玄学**——本册所有行号锚点同时也是埋点位置。

三个经典优化位：

- **预加载裁剪**：`preloaded-classes` 每行一个类，低内存设备（Go 版）走 `--enable-lazy-preload`（`ZygoteInit.java:855`）延迟到首 fork 前；类清单增删直接改变全机内存水位——这是“内存-首帧”权衡的最前线。
- **并行化**：SystemServer 的非关键初始化进 `SystemServerInitThreadPool`（`:899`）；init 侧 ueventd 与主流程并行冷插；APEX 激活（apexd）在 post-fs-data 与服务启动重叠。
- **孵化加速**：USAP 池削 fork 峰值；zygote 的 `gcAndFinalize`（`:897`）减小 COW 复制基数；profile 引导的 ART 预编译让 system_server 的类在 boot.art 里就是机器码。

一个反直觉的事实收尾：**开机最贵的单项常常是 PMS 扫描全部 APK**（`:1233`）与预加载资源——它们都是“用一次性的贵换之后每次的便宜”。理解启动优化，本质是理解 Android 在“首次成本”与“稳态成本”之间做的所有交易。

第十章 设计决策十问

本章把主线上的十个“为什么”整理成问答，每条都可直接追溯到前文锚点。

- **为什么 init 单线程？** 无锁、公平、确定性；代价是吞吐，但启动期没有吞吐需求（第五章）。
- **为什么两阶段 init + 两次 re-exec？** 根文件系统与 SELinux 的鸡生蛋问题，用“每阶段只信任自己布置的世界”解决（第二章）。
- **为什么 rc 是声明式语言？** 同一语法服务开机求值、构建期校验、CTS 检查三处；静态可分析是系统软件的刚需（第四章）。
- **为什么先 reap 再处理其他 fd 事件？** 消除“服务已死未收尸、ctl.start 被拒”竞态（第五章，`init.cpp:1025-1028` 注释）。

- **为什么属性先行？** 触发器、服务的 SELinux 上下文、加密状态全部依赖属性区存在（第三章）。
- **为什么 zygote 禁止起线程？** fork 多线程进程会复制出不一致的锁状态，POSIX 只保证 fork 方线程存活；`startZygoteNoThreadCreation`（`ZygoteInit.java:826`）从源头杜绝。
- **为什么 fork 而不是 exec 应用？** exec 会丢掉预加载的类/资源/ART 状态；fork+写时复制让“加载一次、千次复用”（第六章）。
- **为什么 system_server 由 zygote 第一个 fork 而非 init 拉起？** 它需要整个 Java 框架世界；同时它出生即带 13 项能力、1000 组身份——最小权限在出生时锁定（6.3 节）。
- **为什么服务分三批+boot phase？** 依赖拓扑必须显式化；phase 是全局进度条，服务按阶段就位避免“查无此人”（第七章）。
- **为什么 bootanimation 独立进程？** 它要在 SF 就绪而 AMS 未就绪的窗口里播动画；死了不该影响开机（class core animation 但非 critical）。
- **为什么 SystemUI 由回调启动而不是 rc？** 它是“应用”（有 UI、依赖包扫描与权限），必须在 PMS/AMS 就绪后；rc 世界管不了 Java 依赖。
- **为什么 sys.boot_completed 由 AMS 设置而非 init？** “系统可用”的判据是**框架就绪**（Home 可启动、广播可送达）而不是服务跑起来；只有 AMS 有这个视角（第八章）。

第十一章 深水区：bootloader、fastboot 协议与 OTA 引擎

11.1 bootloader 的两级跳

现代设备两级引导：**первич bootloader（芯片固化 SPL）→ avb 校验二级 bootloader（aboot/ABL，bootable/ 的各平台实现）**→ ABL 读 boot header（书1 §7.1 的 v4）拉内核。ABL 的决策输入：BCB（misc 分区的 boot control block，recovery/ota 想干什么写在里面）+ 按键组合 + charger 状态；输出：normal/bootloader/recovery/fastboot 四种去向。**开机键的"长按进 fastboot"本质是 ABL 在验签前后的一个轮询窗口**。

11.2 fastboot 协议最小集（host 侧源码：system/core/fastboot）

text fastboot 会话（USB bulk 传输，每命令-响应一问一答）

```
host: getvar:max-download-size → 探测可下载块大小
host: download:0x...<data> → 分块推镜像到设备 RAM
host: flash:boot_a → 写入当前槽的 boot 分区
host: getvar:current-slot → A/B 槽位寻址（slot 后缀 _a/_b）
host: set_active:b → 切换启动槽（boot_control HAL 兑现）
host: reboot[-bootloader|fastboot|recovery]
```

fastbootd（userspace 变体，走 adbd 通道）存在的唯一理由：**动态分区**。bootloader 看不懂 super 容器的 metadata，逻辑分区（system/vendor/…）必须由跑起来的系统代刷——这就是"刷 GSI 要先 `fastboot reboot fastboot`"的机制解释。

11.3 GKI 与 boot config

GKI（通用内核镜像）= 内核与 vendor 解耦的镜像版 Treble：boot 分区只装 Google 签名的通用内核，厂商差异走 vendor_boot（vendor ramdisk）、vendor_dlkm（内核模块）与 **bootconfig**（内核 cmdline 的结构化继任者，`androidboot.*` 参数家族）。开机链新增一步：内核启动时读 bootconfig 分区/区域合并参数——排查"参数没生效"时先分清 cmdline 还是 bootconfig 域。

11.4 update_engine：OTA 的执行引擎

`system/update_engine/` 的骨架是**动作流水线**（ActionProcessor 逐 Action 执行）：InstallPlanAction（解 payload manifest）→ FilesystemVerifierAction（校验旧版哈希）→ DownloadAction（HTTP 拉包+DeltaPerformer 增量还原：源块搬运/bz4 解压→写后台槽）→ PostinstallRunnerAction（新槽的OTA脚本）→ boot_control 切槽。失败语义：任意 Action 失败即整单回退，旧槽未动一根毫毛。**payload 签名验两次**（下载前 manifest 签名、块级 hash 树）——与 AVB 构成三明治防御。

> **源码锚点 · 本章锚点**
> system/core/fastboot/：host 工具与 vendor_boot_img 处理
>
bootable/libbootloader/libxbc：bootloader 消息控制库
>
system/update_engine/payload_consumer/：DeltaPerformer 增量还原
build/make/core/Makefile:846-985：dtb/boot 镜像产物规则（书1 §7）

附录 A 观测实操手册

目标	命令	对应源码
看两阶段切换	dmesg 检索 init	init.cpp:839 "init second stage started!"
看触发瀑布	getprop init.svc.* 的时序	service.cpp:684 NotifyStateChange
看预加载	logcat -b events 过滤 preload	ZygoteInit.java:887-890
看服务启动	logcat 过滤 SystemServer	SystemServer.java 各 traceBegin
看 boot phase	dumpsys activity service SystemUIService 等	SystemServiceManager.java:279
权威开机标志	getprop sys.boot_completed	AMS.java:5149
启动时间线	bootchart / perfetto	system/core/init/bootchart.cpp
孵化耗时	logcat -b events 过滤 am_proc_start	Process.start → zygote socket

附录 B 关键文件清单（本册出场人物）

文件	行数	角色
system/core/init/main.cpp	83	四身份入口分发
system/core/init/first_stage_init.cpp	400+	挂分区、切根
system/core/init/init.cpp	1062	第二阶段主体与主循环
system/core/init/parser.cpp	246	rc 解析器框架
system/core/init/action_manager.cpp	110+	事件队列与求值
system/core/init/builtins.cpp	1400+	全部内置命令
system/core/init/service.cpp	871	服务生命周期
system/core/init/property_service.cpp	1396	属性仲裁
system/core/rootdir/init.rc	1300+	开机总谱
frameworks/base/cmds/app_process/app_main.cpp	344	zygote 壳与 JNI 桥
core/java/com/android/internal/os/ZygoteInit.java	1006	Java 世界第一入口
core/java/com/android/internal/os/ZygoteServer.java	600+	孵化循环
services/java/com/android/server/SystemServer.java	3216	服务总装线
services/core/.../SystemServiceManager.java	300+	服务生命周期管理
services/core/.../am/ActivityManagerService.java	18480	调度中枢（本册只涉启动切片）

附录 C 自检题

- 说出 init 的四个身份及触发条件（第二章）。
- `on late-init` 的子触发器顺序为何不能乱？调换 `post-fs-data` 与 `zygote-start` 会怎样？（第四章，提示：/data 加密与 dex）
- zygote 为什么先 `gcAndFinalize()` 再 fork？（第六章，提示：COW 复制的页数）

- system_server 的 Binder 线程上限是多少、在哪个文件哪个常量？（第七章：SystemServer.java:435）
- `sys.boot_completed` 与 `dev.bootcomplete` 谁先谁后、各代表什么？（第八章与附录 A）

附录 D 动手实验（一天量）

实验一“时序考古”：`adb shell dmesg | grep -E "init|zygote"` 找到 "init second stage started!"（init.cpp:839）；`logcat -b events` 过滤 boot_progress_ 事件，把时间戳画成时间线，与本册总时序图对位。

实验二“状态机观察”：每秒一次 `getprop init.svc.zygote` 与 `init.svc.surfaceflinger`，重启设备观察 running→restarting→running 的流转（对应 service.cpp:684 NotifyStateChange 与 Reap :258）。

实验三“剧本推演”：打开 rootdir/init.rc，从 late-init（:516）逐 trigger 朗读子触发器；假设你要加一个“post-data 阶段启动的自研守护进程”，写出它的 rc 片段（service 段 + 放哪个 etc/init 目录 + SELinux 三件套），对照第四章 4.5 的语法表自查。

实验四“bootchart”：按 system/core/init/bootchart.cpp 注释开启采集，导出 PNG，找出占比最大的三段，分别映射到本册的哪个阶段。

实验五“思想实验”：把 zygote-start 移到 post-fs 之前会发生什么？写出三条因果链（提示：/data 加密、preloaded-classes 可达性、dex 位置）。

第十二章 属性触发引擎：init 事件机器的内部视角

> **设计洞察 · 学习目标**
> 1. 说清一次 setprop 之后 init 内部的完整事件流：仲裁 socket→PropertyChanged→事件队列→CheckEvent→ExecuteOneCommand；2. 读懂 Action::CheckEvent 的三个重载与 std::visit 分派；3. 解释 property_triggers_enabled 门闩与 QueueAllPropertyActions 的"补账"机制；4. 能用引擎语义预测任意 `on property:*` 段何时命中

第四章从使用者视角看了 rc 触发器语法与 late-init 瀑布，第三章 3.3 节看到了触发器在 SecondStageMain 里的入队顺序，4.3 节引过 `ExecuteOneCommand` 的"一次一条"节拍。本章换到引擎视角，回答三个问题：**一次 setprop 之后 init 内部发生了什么？** `on property:*` **段到底何时被匹配？开机早期的属性写入会不会"丢失"？** 这段代码是理解 init 一切行为的钥匙，也是"声明式语言如何在运行期求值"的活教材。

12.1 从仲裁 socket 到事件队列：PropertyChanged

事件流的起点在 3.4 节讲过的属性仲裁 socket。`handle_property_set_fd`（`property_service.cpp:527`）完成 SELinux 检查后调 `HandlePropertySet`，后者把值提交进共享属性区（`__system_property_update/add`，`property_service.cpp:195-204`），然后调用回调：

cpp system/core/init/property_service.cpp:209-217（节选）

```
// Don't write properties to disk until after we have read all default
// properties to prevent them from being overwritten by default values.
if (persistent_properties_loaded && StartsWith(name, "persist.")) {
WritePersistentProperty(name, value);
}
// If init hasn't started its main loop, then it won't be handling property changed messages
// anyway, so there's no need to try to send them.
auto lock = std::lock_guard{accept_messages_lock};
if (accept_messages) {
PropertyChanged(name, value);
}
```

`PropertyChanged`（`init.cpp:333`）是属性世界与触发器世界唯一的桥：

cpp system/core/init/init.cpp:333-351（节选+讲解注释）

```
void PropertyChanged(const std::string& name, const std::string& value) {
// 讲解注释①：sys.powerctl 不走事件队列——关机请求立即生效，
// 即使 init 正卡在 wait_for_prop 或 exec 服务上（第五章提过的两种"暂停推进"态）。
// 这解释了为什么"设备再卡也能 setprop sys.powerctl reboot 救活"。
if (name == "sys.powerctl") {
trigger_shutdown(value);
}
// 讲解注释②：门闩！property_triggers_enabled 在 late-init 前后才置 1（见 12.4），
// 此前所有属性写入只更新值、不产生事件——否则开机早期的写入会以
// 随机顺序触发半初始化世界里的 action。
if (property_triggers_enabled) {
ActionManager::GetInstance().QueuePropertyChange(name, value);
WakeMainInitThread(); // 讲解注释③：主线程可能在 epoll.Wait 里睡觉，
} // 写 eventfd 把它叫醒处理新事件
// 讲解注释④：wait_for_prop 命令的等待者在此核对——
// rc 脚本里 wait_for_prop sys.boot_completed 1 之所以能"等到"，
// 就是每次属性变化都顺手问一句"你等的是我吗"。
prop_waiter_state.CheckAndResetWait(name, value);
}
```

注意①与②的对比：**sys.powerctl 是旁路，其余属性都进队列**。这个设计让 init 的所有正常行为都经事件队列串行化（确定性），唯独关机这种"最后一根稻草"操作享受免排队特权。

12.2 事件队列的三种元素与 ExecuteOneCommand

`ActionManager` 的事件队列元素是 `std::variant`：**EventTrigger**（字符串，如 `late-init`）、**PropertyChange**（name/value 对）、**BuiltinAction 指针**（代码运行期临时造的 action，如 3.3 节的 `wait_for_coldboot_done`）。三种事件共用一个队列、一个匹配循环：

cpp system/core/init/action_manager.cpp:43-107（骨架+讲解注释）

```
void ActionManager::QueueEventTrigger(const std::string& trigger) { // :43
auto lock = std::lock_guard{event_queue_lock_};
event_queue_.emplace(trigger); // 讲解注释①：纯边沿事件——入队一次，
} // 错过不补（late-init 永远只来一次）
void ActionManager::QueuePropertyChange(const std::string& name, // :48
const std::string& value) {
auto lock = std::lock_guard{event_queue_lock_};
event_queue_.emplace(std::make_pair(name, value)); // 讲解注释②：属性变化也是事件，
} // 但自带 name/value 供匹配
void ActionManager::QueueAllPropertyActions() { // :53
QueuePropertyChange("", ""); // 讲解注释③：空对=让所有纯属性
} // action 按"当前值"补账（12.4）
void ActionManager::QueueBuiltinAction(BuiltinFunction func, // :57
const std::string& name) {
auto lock = std::lock_guard{event_queue_lock_};
auto action = std::make_unique<Action>(true, nullptr, "<Builtin Action>", 0,
name, std::map<std::string, std::string>{});
action->AddCommand(std::move(func), {name}, 0);
event_queue_.emplace(action.get()); // 讲解注释④：BuiltinAction 入队即持有——
actions_.emplace_back(std::move(action)); // 匹配时按指针相等（:219），不比名字
}
void ActionManager::ExecuteOneCommand() { // :67 主循环每轮调一次（第五章）
{
auto lock = std::lock_guard{event_queue_lock_};
// 讲解注释⑤：只有当没有可执行 action 时才消费事件——
// 对队首事件遍历全部 Action 问"你关心我吗？"，
// 命中的进 FIFO 队列 current_executing_actions_（action_manager.h:57，
// 注意是普通 queue 而非优先级队列：先匹配者先执行）。
while (current_executing_actions_.empty() && !event_queue_.empty()) {
for (const auto& action : actions_) {
if (std::visit([&action](const auto& event) {
return action->CheckEvent(event); }, // 讲解注释⑥：三重载分派
event_queue_.front())) {
current_executing_actions_.emplace(action.get());
}
}
event_queue_.pop(); // 讲解注释⑦：无论命中与否，事件只消费一次
}
}
if (current_executing_actions_.empty()) return;
auto action = current_executing_actions_.front();
if (current_command_ == 0) { // 讲解注释⑧：首条命令前打印 processing action(...)
LOG(INFO) << "processing action (" << action->BuildTriggersString() << ") from ("
<< action->filename() << ":" << action->line() << ")"; // 开机 trace 的"路标"
}
action->ExecuteOneCommand(current_command_); // 讲解注释⑨：一次只执行一条命令
++current_command_;
if (current_command_ == action->NumCommands()) {
current_executing_actions_.pop(); // 讲解注释⑩：执行完毕出队，回⑤继续
current_command_ = 0;
if (action->oneshot()) { // 讲解注释⑪：oneshot action 用完即焚——
auto eraser = [&action](std::unique_ptr<Action>& a) { return a.get() == action; };
actions_.erase(std::remove_if(actions_.begin(), actions_.end(), eraser), actions_.end())
;
} // ↑ 从全集摘除，同名事件再来也不匹配
```

```
}
}
```

设计洞察 · 逐段精读：⑤⑦⑨的三重节拍保证

⑤"空了才消费事件"保证一个事件的所有命中 action 被连续调度（中间不插队）；⑦"只消费一次"保证属性触发天然是**边沿语义**（值不变不再触发）；⑨"一次一条命令"保证 50 条命令的长 action 与新事件交错执行（第四章 4.3）。三个约束合起来，init 才能做到"同一份 rc、同样的属性序列，任何机器上的行为都一致"——这就是第五章说的"确定性"的具体代价与形状。

12.3 匹配规则：CheckEvent 三重载

事件入队只是排队，真正的匹配逻辑在 `Action::CheckEvent`（action.cpp:211-221）：

cpp system/core/init/action.cpp:184-222（节选+讲解注释）

```
// 讲解注释①：属性条件总检查。name/value 可空——空=所有条件按属性区当前值判断；
// 非空=本次刚变化的那一对，用它带来的新值判断（跳过再读一遍属性区）。
bool Action::CheckPropertyTriggers(const std::string& name, // :184
const std::string& value) const {
if (property_triggers_.empty()) return true; // 无属性条件的 action 恒真
if (!name.empty()) {
auto it = property_triggers_.find(name);
if (it == property_triggers_.end()) return false; // 变化的属性与我无关
const auto& trigger_value = it->second;
if (trigger_value != "*" && trigger_value != value) return false;
// 讲解注释②：value="*" 是通配——on property:sys.usb.config=*
// 只要该属性被写入（任意非空值）即命中
}
for (const auto& [trigger_name, trigger_value] : property_triggers_) {
if (trigger_name != name) {
std::string prop_value = android::base::GetProperty(trigger_name, "");
if (trigger_value == "*" && !prop_value.empty()) continue;
if (trigger_value != prop_value) return false; // 讲解注释③：其余条件按"当前值"
} // 判断——这是"电平"语义
}
return true;
}
bool Action::CheckEvent(const EventTrigger& event_trigger) const { // :211
return event_trigger == event_trigger_ && CheckPropertyTriggers();
// 讲解注释④：事件名**精确相等**（无通配）+ 该 action 的全部属性条件当场成立。
// 语法上 on boot && property:ro.debuggable=1 就是"事件命中再查电平"。
}
bool Action::CheckEvent(const PropertyChange& property_change) const { // :215
const auto& [name, value] = property_change;
return event_trigger_.empty() && CheckPropertyTriggers(name, value);
// 讲解注释⑤：纯属性 action 只能被属性变化激活——event_trigger_ 必须为空，
// 且变化的这一对必须本身就是它的触发器之一。事件触发不会"顺手"带动它们。
}
bool Action::CheckEvent(const BuiltinAction& builtin_action) const { // :219
return this == builtin_action; // 讲解注释⑥：指针相等，内置 action 独占匹配
}
```

rc 的触发器行由 ParseTriggers（action_parser.cpp:94-130）解析：&& **是唯一合法连接符**，属性条件进 map，事件条件全段只允许一个。据此得到完整的匹配语义表：

触发器写法	命中条件（引擎语义）	示例
on late-init	事件字符串精确相等；纯边沿，错过不补	on late-init
on property:sys.boot_completed=1	该属性写入此值（边沿）且其余条件按当前值	on property:sys.boot_completed=1
on property:ro.build.type=*	该属性被写入任意非空值	on property:sys.usb.config=*
on boot && property:ro.debuggable=1	事件 boot 命中时电平必须同时为真	on zygote-start && property:x=y
on property:a=b && property:c=d	任一属性变化时，全部条件按当前值成立	组合属性条件

设计洞察 · 设计洞察：边沿+电平的混合模型

属性触发器是"边沿入队、电平匹配"的混合体：变化这一对按新值判（边沿），其余条件按属性区当前值判（电平）。这带来一个必须记住的推论——on property:a=1 && property:b=2 只在 **a 或 b 被写入**时才可能命中；a=b=1 但无人再写属性，它永远安静。事件触发器则是纯边沿：late-init 只有一个瞬间。

12.4 门闩与补账：为什么开机早期的属性不触发 action

property_triggers_enabled 是全局门闩（init.cpp:108，默认 0）。置 1 的唯一路径是一对链式 builtin（init.cpp:529-539）：queue_property_triggers_action（3.3 节 :986 入队，排在 late-init 之后）先入队 property_enable_triggers_action（:532 置 1），再调 QueueAllPropertyActions() 补一次空对事件。

这个两步走回答了本开头的第三个问题：**早期属性写入不会丢**。设想 ro.crypto.state=encrypted 在 init.rc 解析前就由内核 cmdline 写好了——它写入时门闩未开，不会立刻触发 on property:ro.crypto.state=encrypted（4.1 节的 zygote-start 三条件）。门闩打开后的那次 QueueAllPropertyActions 用空对让**所有**纯属性 action 按属性区当前值重新检查一遍——该命中的补上，这叫"补账"。此后世界进入稳态：每次属性写入都边沿触发一次。

设计洞察 · 逐段精读：从引擎视角重看第四章的瀑布

trigger zygote-start 命令的本质是 QueueEventTrigger("zygote-start")；on property:* 的三个 zygote 启动条件（init.rc:1027-1053）靠门闩后的补账被命中；class_start main 之所以能找到 zygote，是因为 Service 对象在解析期就躺在 ServiceList 里（第四章 4.7）。瀑布不是 init 的特殊机制，就是本章这套队列+匹配机器跑出来的观测结果。

12.5 实战：用引擎语义读 init 日志

掌握了引擎，`logcat -d -s init` 里那些 `processing action (...)` 行就变成了可推理的时间线：

text 一台正常设备的 init 处理序列（logcat -d -s init 节选，注释为解读）

```
processing action (early-init) from (/system/etc/init/hw/init.rc:15)
← 3.3 节 :957 入队的第一个事件触发器
processing action (wait_for_coldboot_done) from (<Builtin Action>:0)
← QueueBuiltinAction：指针匹配，阻塞等 ueventd（第二章 2.3.1）
processing action (init) from (/system/etc/init/hw/init.rc:115)
processing action (late-init) from (/system/etc/init/hw/init.rc:516)
← 总谱开始：子触发器按书写顺序 trigger 入队（第四章 4.1）
processing action (early-fs) from (...) / (fs) / (post-fs) / (late-fs) ...
processing action (queue_property_triggers) from (<Builtin Action>:0)
← 门闩置 1 + 补账扫描，属性世界从此通电
processing action (post-fs-data) ... / (zygote-start) ...
processing action (property:ro.crypto.state=encrypted && property:ro.crypto.type=file)
← 补账命中的纯属性 action：ro.crypto.state 早就写好，等的就是这一刻
processing action (boot) from (/system/etc/init/hw/init.rc:1059)
← class_start core / class_start main 在此执行，服务批量孵化
```

三条推论可直接用于排障：某属性 action 没跑，先查该属性在门闩开启后**是否真的被写入过**（边沿）；`on property:a=b && property:c=d` 不命中多半是另一个条件电平不为真；`processing action` 出现两次同一段，说明两个不同事件（如属性写两次）先后命中了它。

源码锚点 · 本章锚点速查

property_service.cpp:209-217 PropertyChanged 调用点 / init.cpp:333-351 PropertyChanged 本体
init.cpp:529-539 门闩与补账 builtin / :986 queue_property_triggers 入队（3.3 节）
action_manager.cpp:43-65 三种入队 / :67-107 ExecuteOneCommand / action_manager.h:57 FIFO
action.cpp:184-208 CheckPropertyTriggers / :211-221 三重载 / action_parser.cpp:94-130 & 语法

第十三章 Zygote 孵化协议与 specialize：一次 fork 的完整解剖

设计洞察 · 学习目标

1. 逐行走通一条孵化请求从 socket 字节流到 ActivityThread.main 的完整旅程；2. 说出孵化协议的核心参数（--setuid/--runtime-flags/--seinfo 等）各自驱动 native 层哪个特化动作；3. 解释 SpecializeCommon 里 setresgid→seccomp→setresuid→SELinux 的顺序为什么不能乱；4. 讲清 system_server 死亡后 zygote 的自毁-重生链条

第六章 6.4/6.5 节看了 runSelectLoop 的外层循环与"分身时刻"，本章下潜一层：进入 ZygoteConnection.processCommand 内部与 native 的 SpecializeCommon，把一次 am_proc_start（附录 A 的事件锚点）在源码层面拆到系统调用粒度。这是本册 C++ 密度最高的一章，也是理解"应用进程为什么天生干净"的最后一块拼图。

13.1 协议分派：processCommand 的一天

孵化请求是一个文本行协议：第一行是参数个数，随后每行一个参数，形如 --setuid=10023，最后一个非 -- 开头的参数是入口类名（通常是 android.app.ActivityThread）。processCommand（ZygoteConnection.java:117）先让 ZygoteArguments.getInstance（ZygoteArguments.java:260，解析在 :274 parseArgs）吃掉字节流，然后按命令类型分派：

java frameworks/base/core/java/com/android/internal/os/ZygoteConnection.java:117-284（骨架+讲解注释）

```
Runnable processCommand(ZygoteServer zygoteServer, boolean multipleOK) {
ZygoteArguments parsedArgs;
try (ZygoteCommandBuffer argBuffer = new ZygoteCommandBuffer(mSocket)) {
while (true) {
parsedArgs = ZygoteArguments.getInstance(argBuffer); // :123 读并解析协议
if (parsedArgs == null) { // :129 流结束（EOF）
isEof = true;
return null; // 会话收尾（6.5 节②的下游）
}
// 讲解注释①：协议分派——孵化请求之外还有一堆"管理命令"：
// mBootCompleted（第八章 :5085 打到这里）/ mAbiListQuery / mPidQuery / mPreloadDefault
if (parsedArgs.mBootCompleted) { handleBootCompleted(); return null; } // :141
if (parsedArgs.mAbiListQuery) { handleAbiListQuery(); return null; } // :145
if (parsedArgs.mPidQuery) { handlePidQuery(); return null; } // :149
if (parsedArgs.mPreloadDefault){ handlePreload(); return null; } // :159
// 讲解注释②：安全闸门第一道——**普通客户端禁止指定 capabilities**，
// 能力位只能由 native 层按 uid 计算（13.3），app 数据线上写来的不算数。
if (parsedArgs.mPermittedCapabilities != 0
|| parsedArgs.mEffectiveCapabilities != 0) {
throw new ZygoteSecurityException("Client may not specify capabilities"); // :192
}
// 讲解注释③：安全闸门第二道——只有 SYSTEM_UID 的客户端才能指定 uid/gid/gids
Zygote.applyUidSecurityPolicy(parsedArgs, peer); // :199 否则强制继承 peer 身份
Zygote.applyInvokeWithSecurityPolicy(parsedArgs, peer); // invoke-with 同样限权
Zygote.applyDebuggerSystemProperty(parsedArgs); // :202 ro.debuggable 传染
Zygote.applyInvokeWithSystemProperty(parsedArgs);
...
int[] fdsToClose = { -1, -1 }; // :251
// 讲解注释④：fdsToClose 收集两个 socket fd——native fork 后子进程用
// dup2(fd, /dev/null) 抹掉它们，防止孩子继续偷听孵化信道（fork 泄漏经典题）
if (parsedArgs.mInvokeWith != null || parsedArgs.mStartChildZygote
|| !multipleOK || peer.getUid() != Process.SYSTEM_UID) {
pid = Zygote.forkAndSpecialize(parsedArgs.mUid, parsedArgs.mGid, // :253
parsedArgs.mGids, parsedArgs.mRuntimeFlags, rlimits,
parsedArgs.mMountExternal, parsedArgs.mSeInfo, parsedArgs.mNiceName,
fdsToClose, fdsToIgnore, ...);
try {
if (pid == 0) { // :263 子进程——"分身时刻"
```

```
zygoteServer.setForkChild(); // :265 mIsForkChild=true（6.5 节③）
zygoteServer.closeServerSocket(); // :267 关孵化 socket
return handleChildProc(parsedArgs, childPipeFd, parsedArgs.mStartChildZygote
);
} else { // :271→514→zygoteInit→applicationInit；父进程收尾
IoUtils.closeQuietly(childPipeFd); // :276
handleParentProc(pid, serverPipeFd); // :278 读子进程状态、维护 USAP
return null; // 继续守循环
}
} finally {
IoUtils.closeQuietly(childPipeFd); // :281 管道两边各自关干净
IoUtils.closeQuietly(serverPipeFd);
}
} else {
// 讲解注释⑤：system_server 高峰期直发请求走 forkSimpleApps 快车道
// （批量 fork，USAP 补水同款机制）；fork 失败/管理命令则回普通路径
ZygoteHooks.preFork(); // :286
Runnable result = Zygote.forkSimpleApps(argBuffer, ...); // :287
...
}
}
}
}
```

设计洞察 · 逐段精读：协议即安全边界

孵化 socket 的 peer 是内核背书的（第三章 3.4 节同款 SO_PEERCRED），`peer.getUid()` 不可伪造。于是协议解析层天然分层：管理命令只有特权 peer 能用；uid/gid 只认 SYSTEM_UID；capabilities 一律 native 重算。**把安全检查放在参数解析之后、fork 之前**，意味着恶意参数最多失败，永远不可能"先 fork 再被发现"。

13.2 协议参数 → 特化动作对照表

`ZygoteArguments` 的每个字段都对应 native 层一个精确的特化步骤。这张表是"文本参数如何变成进程属性"的地图：

协议参数	ZygoteArguments 字段	native 特化动作（SpecializeCommon 内）
--setuid=10023	mUid	:1645 setresuid(10023,10023,10023)
--setgid=10023	mGid	:1632 setresgid（先于 setuid！）
--setgroups=3003,...	mGids	:531 SetGids（附加组，如 inet 3003）
--runtime-flags=N	mRuntimeFlags	MTE/堆零初始化/JDWP/GWP-ASan 开关（:1681-1730）
--mount-external=N	mMountExternal	:1578 MountEmulatedStorage（存储沙盒挂载）
--seinfo=default:targetSdkVersion=33...	mSeInfo	:1723 selinux_android_setcontext（定 SELinux 域）

协议参数	ZygoteArguments 字段	native 特化动作（SpecializeCommon 内）
--nice-name=com.example.app	mNiceName	:1730 SetThreadName（ps 可见名）
--capabilities=...（仅 system_server fork 用）	mPermittedCapabilities	:1713 SetCapabilities
--rlimit=r,c,m	...	:1610 SetRLimits
--invoke-with=...	mInvokeWith	走 WrapperInit.execApplication（:528）
行尾类名 android.app.ActivityThread	mRemainingArgs	handleChildProc→applicationInit 反射入口

13.3 SpecializeCommon：两次系统调用之间的沙盒工程

`forkAndSpecialize` 的 JNI 落点是 `com_android_internal_os_Zygote.cpp`：先 `zygote::ForkCommon`（:2033，fork 本体），子进程返回后进 `SpecializeCommon`（:1544）。后者要在"从 fork 继承的 zygote 全权身份"降到"目标 App 的最小身份"，中间是一串**顺序严格**的变换：

cpp frameworks/base/core/jni/com_android_internal_os_Zygote.cpp:1544-1795（骨架+讲解注释）

```
static void SpecializeCommon(...) {
const char* process_name = is_system_server ? "system_server" : "zygote";
auto fail_fn = std::bind(ZygoteFailure, env, process_name, ...); // 失败即 abort 整个 zygote
...
// 讲解注释①：能力准备。uid!=0 时先开 PR_SET_KEEPCAPS——让 setuid 之后
// permitted/effective 能力位不被内核清空，才能活着走完下面的流程。
if (uid != 0) EnableKeepCapabilities(fail_fn); // :1562
SetInheritable(permitted_capabilities, fail_fn); // :1567
DropCapabilitiesBoundingSet(fail_fn); // :1569 bounding set 收缩
// 讲解注释②：存储沙盒。挂载模式（NONE/PASS_THROUGH/DEFAULT/READABLE/...）决定
// /storage 视图；随后 ensureInAppMountNamespace 让 App 独占 mount namespace。
MountEmulatedStorage(uid, mount_external, ..., fail_fn); // :1578
ensureInAppMountNamespace(fail_fn); // :1581
if (mount_data_dirs) { // :1585
isolateAppData(...); // tmpfs 覆盖 + bind mount：每个 App 只能看见自己的 /data 数据
isolateJitProfile(...);
}
// 讲解注释③：cgroup 与附加组、rlimits——App 的"资源户口"在出生时登记。
if (!is_system_server && getuid() == 0) createProcessGroup(uid, getpid()); // :1600
SetGids(env, gids, is_child_zygote, fail_fn); // :1609
SetRLimits(env, rlimits, fail_fn); // :1610
if (is_system_server) { /* 预取 system_server classloader，为 SELinux 域切换铺路 */ } // :1626
// 讲解注释④：降 gid 必须在降 uid **之前**——丢掉 root uid 后就再无权限改 gid。
if (setresgid(gid, gid, gid) == -1) fail_fn(...); // :1632
// 讲解注释⑤：seccomp 过滤器必须在**还有 CAP_SYS_ADMIN** 时安装
//  （丢 uid 后无权装 filter）；源码注释明言改用 PR_SET_NO_NEW_PRIVS 会
// 破坏 SELinux 域切换（b/71859146）——顺序是权衡后的唯一解。
SetUpSeccompFilter(uid, is_child_zygote); // :1639
SetSchedulerPolicy(fail_fn, is_top_app); // :1643 丢权限前设 cgroup
if (setresuid(uid, uid, uid) == -1) fail_fn(...); // :1645 ★ 身份切换完成 ★
... // dumpable 处理（:1649）、MTE/堆零初始化/GWP-ASan（:1681-1730）
```

```
SetCapabilities(permitted_capabilities, effective_capabilities, ...); // :1713
// 讲解注释⑥：SELinux 域切换放在最后——策略要按**最终 uid + seinfo** 算域；
// App 域形如 u:r:untrusted_app_33:s0:c512,c768（seinfo 决定）。
if (selinux_android_setcontext(uid, is_system_server,
se_info_ptr, nice_name_ptr) == -1) fail_fn(...); // :1723
SetThreadName(nice_name_ptr ?: process_name); // :1730 ps 名在此定格
UnsetChldSignalHandler(); // :1736 孩子不再收尸
if (is_system_server) { // :1747
/* PostForkSystemServerHooks + 显式 setcon("u:r:system_server:s0") */
}
if (is_child_zygote) initUnsolSocketToSystemServer(); // :1781 app zygote 上行通道
env->CallStaticVoidMethod(gZygoteClass, gCallPostForkChildHooks, ...); // :1786 ART 收尾
setpriority(PRIO_PROCESS, 0, PROCESS_PRIORITY_DEFAULT); // :1792 优先级归位
}
```

fork 本体的清洁工作同样值得驻足（ForkCommon，:2033-2114）：fork 前 `BlockSignal(SIGCHLD)` 防止日志 fd 在复制途中被重开；`__android_log_close()` 关掉共享日志 fd；`FileDescriptorTable`（首次 fork 建、此后 Restat 校验）保证**每次 fork 时的 fd 表形状一致**——否则 fd 泄漏会让"同 zygote 孵出的进程"行为分岔；`mallopt(M_PURGE)` 归还空闲页，减少 fork 后子进程因 malloc 元数据写而触发的 COW 复制（第六章 6.6 的定量表的隐藏项）。

> **设计洞察 · 设计洞察：顺序即安全**
>
> SpecializeCommon 的每一步都在收窄一个权限窗口：KEEPCAPS→bounding set→mount ns→gid→seccomp→uid→SELinux。任何两步对调都会失败或留洞（丢 uid 后装不了 seccomp；切域后改不了 mount）。**系统编程里"顺序"不是风格问题，是正确性问题**——这条 30 行的序列是给 Linux 能力模型学习者最好的练习题。

13.4 端到端串讲：一条孵化请求的一生

把六章的碎片拼成一条完整时间线（对照第一章图 boot-timeline 与 6.4 节）：

text am_proc_start 从字节到 ActivityThread.main（text 时间线，行号=源码锚点）

```
system_server: ActivityThread/AMS → ProcessList.startProcessLocked → Process.start
→ ZygoteProcess.send ZygoteArguments（写 socket：--setuid=10023 --runtime-args ...）
zygote: ZygoteServer.runSelectLoop poll 返回（6.5 节①②）
→ processCommand :117 → ZygoteArguments.parseArgs :274
→ 安全闸门 :192/:199 → Zygote.forkAndSpecialize（Zygote.java）
→ JNI: ForkCommon :2033（阻塞 SIGCHLD→清日志 fd→FD 表→M_PURGE→fork()）
→ 子进程：SpecializeCommon :1544（13.3 的十步序列）
→ handleChildProc :514 → closeSocket :522
→ ZygoteInit.zygoteInit :980（commonInit / nativeZygoteInit / applicationInit）
- nativeZygoteInit → app_main.cpp onZygoteInit → Binder 线程池诞生（第三册）
→ applicationInit → findStaticMain → ActivityThread.main()
system_server: handleParentProc :278 读子进程状态 → logcat am_proc_start 落笔（附录 A）
```

最后补一块拼图——**system_server 死了 zygote 怎么办**：nativeForkSystemServer（:2194）在父进程登记 `gSystemServerPid`（:2209）并立刻 `waitpid(WNOHANG)` 复核（:2214，防"注册前就崩"的窗口）；此后 zygote 的 SIGCHLD 处理器（:455-462）一旦发现死者是 system_server，就 `kill(getpid(), SIGKILL)` **自杀**——让 init 以 zygote 的 critical 服务身份重启它（下一章的熔断器）

，system_server 再由新 zygote 重新 fork。这是一条精心设计的自毁-重生链：**没有一个组件"知道"如何重建框架，但每个组件都知道如何让自己可被重建**。

> **源码锚点 · 本章锚点速查**
> ZygoteConnection.java:117 processCommand / :192 capabilities 闸门 / :199 uid 安全策略
:253 forkAndSpecialize / :263-280 子父分岔 / :514 handleChildProc / :538 zygoteInit
ZygoteArguments.java:260/274 协议解析
com_android_internal_os_Zygote.cpp:2033 ForkCommon / :1544 SpecializeCommon
:1632 setresgid / :1639 seccomp / :1645 setresuid / :1723 SELinux setcontext
:455-462 system_server 死亡→zygote 自杀 / :2209-2217 gSystemServerPid 登记

第十四章 Watchdog：system_server 的心电图

> **设计洞察 · 学习目标**
> 1. 说清 Watchdog 的探针原理：为什么必须在被监控线程自己的 Looper 上跑；2. 读懂 HandlerChecker 四态机（COMPLETED/WAITING/WAITED_HALF/OVERDUE）与 60 秒判据；3. 掌握 watchdog 触发后的完整链条：半超时 dump→kill system_server→zygote 自毁→init 重生；4. 会区分 watchdog（死锁）与 lmkd（内存压力）两种完全不同的"系统杀我"

第七章的 Bootstrap 批清单里，Watchdog 排在第一（`SystemServer.java:1058`）——注释写着"Start the watchdog as early as possible so we can crash the system server if we deadlock during early boot"。它必须先于一切被监控者存在，这是"看门狗要在犬舍盖好前进驻"的顺序依赖。本章解剖这台机器：它如何探测"某条线程堵死了"，超时后如何留下最大信息量再杀进程，以及它和第十五章 init 熔断器如何构成两级防线。

14.1 探针原理：把自己投进被监控线程的队列

Watchdog 是 system_server 内的一个独立线程（`Watchdog` 构造器 `:393 new Thread(this::run, "watchdog")`）。它监控两类对象：

- **Handler 线程**（`addThread，:562`）：构造器默认注册 7 条共享线程——foreground（`mMonitorChecker` 兼任）、main、ui、i/o、display、animation、surface animation（`:404-424`）；
- **Monitor 接口**（`addMonitor，:559`）：实现了 `Watchdog.Monitor` 的服务（AMS、PMS、WMS 等）把自己挂到 `mMonitorChecker` 上，`monitor()` 方法会在**前台线程**里被依次回调——服务在

monitor() 里读自己的关键状态，读得动=服务没死锁。

探测的手法朴素而天才：HandlerChecker.scheduleCheckLocked（:256）把 checker 自己 mHandler.postAtFrontOfQueue(this) 投到被监控线程的 Looper 队首。**一条线程若能执行这个探针，它的消息队列就是活的**——这种"健康"只有线程自己能证明，任何外部观测（活着、有 CPU）都看不到"队列堵死"。四态判据在 getCompletionStateLocked（:286-298）：

状态	判据	含义
COMPLETED	mCompleted=true	探针已执行完，线程健康
WAITING	latency 小于超时的一半	还在跑，不慌
WAITED_HALF	latency 介于一半与全超时之间	可疑，先取证
OVERDUE	latency 超过全超时	判死，准备开杀

14.2 主循环逐行：run() 的心跳

java frameworks/base/services/core/java/com/android/server/Watchdog.java:716-853（骨架+讲解注释）

```
private void run() {
boolean waitedHalf = false;
while (true) {
List<HandlerChecker> blockedCheckers = Collections.emptyList();
String subject = "";
boolean allowRestart = true; // :722 (debugger 保护见讲解注释④)
boolean doWaitedHalfDump = false;
final ArrayList<Integer> pids;
final long watchdogTimeoutMillis = mWatchdogTimeoutMillis, // :727 默认 60s (:97)
checkIntervalMillis = watchdogTimeoutMillis / 2; // :728 检查节拍=30s
synchronized (mLock) {
long timeout = checkIntervalMillis;
for (int i=0; i<mHandlerCheckers.size(); i++) { // :734
// 讲解注释①：每轮给所有 checker 重排探针——已完成（mCompleted）的才真正入队
HandlerCheckerAndTimeout hc = mHandlerCheckers.get(i);
hc.checker().scheduleCheckLocked(hc.customTimeoutMillis()
.orElse(watchdogTimeoutMillis * Build.HW_TIMEOUT_MULTIPLIER)); // :738
}
...
// 讲解注释②：uptimeMillis 而非 elapsedRealtime——睡着的时间不计入（:746-749）
long start = SystemClock.uptimeMillis(); // :750
while (timeout > 0) { // :751 醒来复核
mLock.wait(timeout); timeout = checkIntervalMillis - (SystemClock.uptimeMillis() - s
tart); // :756/:764
}
final int waitState = evaluateCheckerCompletionLocked(); // :767 取最大状态
if (waitState == COMPLETED) { waitedHalf = false; continue; } // :768
else if (waitState == WAITING) { continue; } // :772 还在窗口内
else if (waitState == WAITED_HALF) { // :775 半超时=30s
if (!waitedHalf) {
Slog.i(TAG, "WAITED_HALF"); waitedHalf = true; // :777-778
blockedCheckers = getCheckersWithStateLocked(WAITED_HALF);
subject = describeCheckersLocked(blockedCheckers);
```

```
pids = new ArrayList<>(mInterestingJavaPids);
doWaitedHalfDump = true;
} else continue; // :785 已 dump 过
} else { // :787 OVERDUE!
blockedCheckers = getCheckersWithStateLocked(OVERDUE); // :789
subject = describeCheckersLocked(blockedCheckers);
allowRestart = mAllowRestart; // :791
pids = new ArrayList<>(mInterestingJavaPids); // :792 连带 native 侧
}
}
// 讲解注释③：无论半超时还是判死，第一件事都是**留证据**：dumpStackTraces
// 抓全部线程栈（:855 logWatchog）+ dropbox（半超时 tag=pre_watchdog，判死 tag=watchdog）
logWatchog(doWaitedHalfDump, subject, pids); // :802
if (doWaitedHalfDump) continue; // :804 只取证不杀
// 讲解注释④：IActivityController（monkey 等测试框架）可说"继续等"（:819）；
// 调试器连着或 AMS setAllowRestart(false) 时也只喊不动手（:829-838）
if (mController != null && mController.systemNotResponding(subject) >= 0) continue;
if (!allowRestart) {
Slog.w(TAG, "Watchdog is *not* killing the system process");
} else {
Slog.w(TAG, "*** WATCHDOG KILLING SYSTEM PROCESS: " + subject); // :840
WatchdogDiagnostics.diagnoseCheckers(blockedCheckers); // :841 打印阻塞点
if (!Build.IS_USER && isCrashLoopFound()) breakCrashLoop(); // :843
Process.killProcess(Process.myPid()); // :847
System.exit(10); // :848
}
}
}
```

设计洞察 · 逐段精读：为什么是 30/60/取证两段式

60 秒判据（:97 DEFAULT_TIMEOUT，user 版 60s、调试版 10s，Settings.Global.WATCHDOG_TIMEOUT_MILLIS 可动态调整 :507-511）分两半走：前 30 秒只 dump 不杀（WAITED_HALF），给"慢而未死"的服务一次自证机会；后 30 秒仍卡才判死。**先取证后执行**让每次 watchdog 都自带现场：/data/system/dropbox/watchdog@* 里的 trace 就是从 :802 写下的。

14.3 触发之后的完整链条：从 kill(10) 到新桌面

Watchdog 杀的是 system_server 自己（:847-848），之后的链路在第十三章 13.4 已经预演：

- system_server 死亡 → zygote 的 SIGCHLD 处理器发现死者是 gSystemServerPid → kill(getpid(), SIGKILL) 自毁（com_android_internal_os_Zygote.cpp:455-462）；
- init Reap zygote：它带 critical 标记 → 触发重启策略（第十五章），延迟数秒重启 zygote；
- 新 zygote 重新走 preload（此时大部分类已在 page cache 与 boot.art 里，快得多）→ 重新 forkSystemServer → SystemServer 三阶段重跑（第七章）→ sys.boot_completed 重新置位（第八章）。

对用户的体感：桌面闪一下回到锁屏或重启动画，已打开的应用被杀（zygote 自毁杀掉全部 App 进程）。这也解释了一个运维常识：**system_server 重启 ≠ 关机**，但代价接近一次不完整重启。

14.4 与 lmkd 的边界：两种"杀"，别混淆

初学者常把 watchdog 与 lmkd（lowmemorykiller 守护进程）混为一谈，源码层面它们的唯一交集是共享日志通道：

维度	Watchdog	lmkd
所在进程	system_server 内部线程	独立 native 进程（`system/memory/lmkd`）
杀谁	只杀 system_server 自己	只杀 App/后台进程
判据	Handler 线程消息队列超时	内存压力水位（minfree/psi）
配置	WATCHDOG_TIMEOUT_MILLIS	ro.lmk.* 属性族
日志特征	"WATCHDOG KILLING SYSTEM PROCESS"/"GOODBYE!"	"lowmemorykiller: Killing ..."
用户体感	桌面/框架整体重启	单个后台应用消失

设计洞察 · 设计洞察：看门狗为什么不能装在 init 或内核里

"进程活着"用 kill(pid,0) 就能从外面测，但"消息队列堵死"只有队列自己知道——探针 `postAtFrontOfQueue` 必须跑在被监控线程的 Looper 上（14.1）。这决定了 watchdog 必须与被监控者同进程。代价是它与 system_server 共生共死，于是才需要第十三章的自毁链与第十五章的 init 熔断器兜底——**三层防线各管一层，每层的观测点都在它能看到的地方**。

源码锚点 · 本章锚点速查

Watchdog.java:97 DEFAULT_TIMEOUT 60s / :100-103 四态常量 / :228 HandlerChecker
:256 scheduleCheckLocked / :286-298 四态判据 / :393-427 构造器与 7 线程注册
:562 addThread / :636 evaluateCheckerCompletionLocked / :716-853 run() 主循环
:802 logWatchog 取证 / :840-848 判死三连 / SystemServer.java:1058-1059 最早注册

第十五章 服务重启策略：init 的熔断器

设计洞察 · 学习目标

1. 说出服务死亡到重启的完整信号链：SIGCHLD→signalfd→ReapAnyOutstandingChildren→Service::Reap；2. 读懂 Reap 里的 flags 状态机（oneshot/disabled/critical/restarting）；3. 掌握 4 分钟 4 次熔断判据与 critical window/target 参数；4. 能区分 init 熔断（管崩溃）与 Watchdog（管卡死）两级防线的分工

第五章主循环的第②步 HandleProcessActions 只回答"何时把 restarting 的服务再拉起"；**"要不要重启、多久内允许死几次、超了怎么办"**的决策全部在 Service::Reap。它是 init 对守护进程的保险合同，也是 init.svc.* 状态机（第四章 4.4）的另一面。本章逐行走读这份合同——它是排障手册（文末）"服务死循环重启"场景的理论基础。

15.1 死亡通知链：从内核到 Reap

服务进程死亡 → 内核给 init 发 SIGCHLD → InstallSignalFdHandler（第三章 3.2）把它变成 epoll 事件 → 主循环**先收尸**（第五章 :1025-1028 的竞态注释）→ ReapAnyOutstandingChildren → waitid 拿到每个死者的 siginfo → 对应服务的 Reap。整条链是单线程的，因此 Reap 里读改 flags_ 从不需要加锁——单线程设计（第五章）的红利兑现处。

15.2 Reap 逐段精读：flags 状态机与熔断判据

cpp system/core/init/service.cpp:258-359（骨架+讲解注释）

```
void Service::Reap(const siginfo_t& siginfo) {
// 讲解注释①：杀进程组——oneshot 且非重启场景是唯一例外（源码注释记录了 2007~R 的行为变更）
if (!(flags_ & SVC_ONESHOT) || (flags_ & SVC_RESTART)) {
KillProcessGroup(SIGKILL, false); // :260
} else { KillProcessGroup(SIGKILL, true); } // :262 带 warning 上报
// 讲解注释②：socket 回收——persist socket 保留，其余 /dev/socket/<name> unlink（:271-277）
for (const auto& socket : sockets_) {
if (socket.persist) continue;
unlink((ANDROID_SOCKET_DIR "/" + socket.name).c_str()); // :275
}
...
// 讲解注释③：reboot_on_failure 选项——异常退出（信号或非零码）直接关机
if ((siginfo.si_code != CLD_EXITED || siginfo.si_status != 0) && on_failure_reboot_target_) {
trigger_shutdown(*on_failure_reboot_target_); // :284-288
}
...
if (flags_ & SVC_TEMPORARY) return; // :297 exec 型服务到此结束
pid_ = 0; flags_ &= (~SVC_RUNNING); // :299-300
// 讲解注释④：oneshot 的自然归宿是 disabled——跑完一次就不再自动拉起
if ((flags_ & SVC_ONESHOT) && !(flags_ & SVC_RESTART) && !(flags_ & SVC_RESET)) {
flags_ |= SVC_DISABLED; // :305
}
// 讲解注释⑤：disabled/reset 服务死了就是死了，状态置 stopped 由人接管
if (flags_ & (SVC_DISABLED | SVC_RESET)) {
NotifyStateChange("stopped"); // :311 → init.svc.X=stopped
return;
}
...
// 讲解注释⑥：★ 熔断判据 ★——critical 或"含可升级 APEX 组件"的服务：
// fatal_crash_window_（默认 4 分钟，service.h:172）内死超 4 次或开机完成前就死。
boot_clock::time_point now = boot_clock::now(); // :323
if (((flags_ & SVC_CRITICAL) || is_process_updatable) && !(flags_ & SVC_RESTART)) {
bool boot_completed = GetBoolProperty("sys.boot_completed", false);
if (now < time_crashed_ + fatal_crash_window_ || !boot_completed) { // :328
if (++crash_count_ > 4) { // :330 第 5 次进入
```

```
auto exit_reason = boot_completed
? "in " + std::to_string(fatal_crash_window_.count()) + " minutes"
: "before boot completed";
if (flags_ & SVC_CRITICAL) {
if (!GetBoolProperty("init.svc_debug.no_fatal." + name_, false)) {
SetFatalRebootTarget(fatal_reboot_target_);
LOG(FATAL) << "critical process '" << name_
<< "' exited 4 times " << exit_reason; // :335
} // LOG(FATAL) 触发 init 自身 abort → 按目标重启整机
} else {
LOG(ERROR) << "process with updatable components '" << name_
<< "' exited 4 times " << exit_reason; // :343-344
SetProperty("sys.init.updatable_crashing_process_name", name_); // :346
SetProperty("sys.init.updatable_crashing", "1"); // 通知 update_verifier/apexd
}
}
} else {
time_crashed_ = now; crash_count_ = 1; // :351-352 窗口重开重新计数
}
}
flags_ &= (~SVC_RESTART);
flags_ |= SVC_RESTARTING; // :353 状态机进入 restarting
onrestart_.ExecuteAllCommands(); // :356 执行 onrestart 挂钩
NotifyStateChange("restarting"); // :358 → init.svc.X=restarting
}
```

拉起的时机由主循环的调度器给出（第五章第②步）：

cpp system/core/init/init.cpp:352-379（节选+讲解注释）

```
static std::optional<boot_clock::time_point> HandleProcessActions() {
std::optional<boot_clock::time_point> next_process_action_time;
for (const auto& s : ServiceList::GetInstance()) {
...
if (!(s->flags() & SVC_RESTARTING)) continue; // 只看 restarting 的
// 讲解注释①：重启时刻=上次启动时间+restart_period（默认 5 秒，service.h:214；
// 可用 restart_period 选项调大——对"起来又死"的服务退避）
auto restart_time = s->time_started() + s->restart_period();
if (boot_clock::now() > restart_time) {
if (auto result = s->Start(); !result.ok()) {
LOG(ERROR) << "Could not restart process '" << s->name()
<< "': " << result.error(); // 讲解注释②：失败不抛、只记
} // 下一轮循环再试
} else {
// 讲解注释③：还没到点——把"最早要醒来的时刻"交给主循环
if (!next_process_action_time || restart_time < *next_process_action_time) {
next_process_action_time = restart_time; // epoll 按它定时（第五章③）
}
}
}
return next_process_action_time;
}
```

15.3 重启策略矩阵：一张表记住 flags

rc 配置	死亡后行为	熔断/上限	典型例子
（默认）	restarting→5 秒后重启	无限重启（每次退避 5s）	surfaceflinger 等
critical	同上，且 4 分钟 4 次或开机前崩溃→init LOG(FATAL) 重启整机	window=N 可调，target= 指定重启目标	zygote、servicemanager、vold
oneshot	退出即 disabled，不自动重启	无（可被 restart 唤醒）	bootanim、一次性的 setup
disabled	不随 class_start 启动，ctl.start 才跑	死亡后 stopped	按需服务
critical window=off	熔断关闭（debug 用）	无	临时诊断
reboot_on_failure	异常退出→trigger_shutdown	立即	加密关键步骤

`critical` 的参数解析在 `ServiceParser::ParseCritical`（`service_parser.cpp:96-132`）：只接受 **window=<分钟>** 与 **target=<重启目标>** 两个键值（`window=off` 直接关熔断），且 `critical` 与 `oneshot` 互斥（`:682`，构建期 `host_init_verifier` 就会拦下）。zygote 的定义（`init.zygote64.rc:17`）是活标本：`critical window=${zygote.critical_window.minute:-off} target=zygote-fatal`——**默认 window=off（熔断关闭），target=zygote-fatal 指定崩溃目标的兜底**；`init.rc:1249` 的 ueventd（`critical+shutdown critical`）才是裸 `critical` 的样本。

> **设计洞察 · 设计洞察：两级看门狗与"失败方式的选择"**
> framework 级 Watchdog（第十四章）管"**卡死但活着**"——只能靠杀进程解套；init 级 critical 管"**反复崩溃**"——重启无法挽回时，熔断器选择**换一种失败方式**：LOG(FATAL) 让 init 自身 abort、按 target 重启到 bootloader/recovery，把日志留给下次开机。而 `sys.init.updatable_crashing=1` 是第三条路：不重启整机，通知 apexd/update_verifier 回滚可升级模块。**好的容错设计不是"绝不失败"，是为每种失败预定义一种代价最小的结局**。

> **源码锚点 · 本章锚点速查**
> service.cpp:258-359 Reap / :260 杀进程组 / :271 socket 回收 / :305 oneshot 归宿
:324-350 熔断判据 / :335 LOG(FATAL) / :346-347 updatable_crashing / :356 onrestart
service.h:172 fatal_crash_window_=4min / :214 restart_period_=5s
init.cpp:352-379 HandleProcessActions / service_parser.cpp:96-132 ParseCritical / :682 互斥

教科书习题集（分级）

第一级：简答题（检验概念理解）

1. init 的四个身份是什么？各由什么触发？

2. 为什么 init 要分两阶段执行？每阶段做什么？

3. rc 语言的两种触发器是什么？各在什么时候触发？

4. Zygote 为什么禁止起线程？这个约束解决了什么问题？

5. forkSystemServer 传了哪些"出生证明"参数？

6. SystemServer 为什么分三批启动服务？每批的特点是什么？

7. boot phase 的 550 和 600 分别放行什么？

8. BOOT_COMPLETED 和 sys.boot_completed 谁先谁后？各代表什么？

第二级：分析题（检验机制理解）

9. **追踪时序**：画出从 kernel 拉起 init 到 zygote socket 可用的时间线，标出每个阶段的关键属性变化。

10. **COW 分析**：fork 后 App A 和 App B 同时使用 Button 类的 onClick 方法——它们的 onClick 代码物理上是同一份内存吗？为什么？

11. **失败推演**：如果 init.rc 的 late-init 触发链中 zygote-start 被移到 post-fs 之前，会发生什么？给出三条因果链。

12. **并发对照**：init 是单线程事件驱动，system_server 是几十个 binder 线程+Handler 线程池——为什么两个进程选择了截然不同的并发模型？

13. **顺序依赖**：如果 PackageManagerService 在 AMS 之前启动（调换顺序），哪个服务会先崩溃？为什么？

第三级：设计题（检验系统思维）

14. **设计决策**：假设 Google 要把 preload 从"全量预载"改为"按需预载+云端 profile"（Android 12+ 的方向），你会怎么设计这个系统的接口？（提示：profile 的生成/下载/应用/回退）

15. **性能方案**：一台低端设备（1GB 内存）开机 40 秒，你要在两周内优化到 25 秒，给出分阶段方案（第一周度量+定位，第二周定向优化）。

16. **架构演进**：如果要支持"system_server 运行时重启不重启 zygote"（Android 已在做），需要改动哪些机制？列出三个必须解决的核心问题。

第四级：源码走读题（检验白盒能力）

17. **属性引擎**：一次 `setprop vendor.example.state ready` 之后，init 内部的完整事件流是什么？如果这次写入发生在 `queue_property_triggers` 执行之前，会发生什么？（第十二章）

18. **孵化协议**：`ZygoteConnection.processCommand` 收到客户端（system_server）传来的 `--capabilities=...` 参数会被如何处理？为什么这样设计？（第十三章）

19. **watchdog 判据**：某条被监控线程 60 秒内"活着但消息队列堵死"，Watchdog 在第 30 秒与第 60 秒分别做什么？取证文件落在哪？（第十四章）

20. **熔断参数**：把 zygote 的 rc 改成 `critical window=8 target=zygote-fatal` 会改变什么？写出 critical 熔断与 Watchdog 60 秒两道防线在时间线上的关系。（第十五、十四章）

参考答案要点

1. ①默认 FirstStageMain ②subcontext ③selinux_setup ④second_stage；由 argv[1] 和 argv[0] 区分。

10. 是同一份物理内存——onClick 是方法代码（存在 boot.art 的只读映射中），COW 页未被写入 → 共享。只有 App 各自的 Button 实例（堆对象）是独立的。

11. ①zygote 要读 /data 的 dex/资源 → post-fs 未完成=不可用 ②ro.crypto.state 未定=无法选触发分支 ③apexd 未激活=ART APEX 不可用。

15. 度量：bootchart+logcat boot_progress 分段。定位：zygote preload 慢（低端设备 IO 慢）→ 裁剪+lazy preload。system_server 慢 → PMS 并行+减 priv-app 数量。动画 → 提前放行。

2. 阶段一在 ramdisk 临时根上完成"拼出可启动的根"（first_stage_mount 挂 /system 等）；阶段二在真实根上装属性区、SELinux 策略、rc 求值与主循环；中间两次 re-exec 让每个阶段只信任自己亲手布置的世界（第二章）。

3. 事件触发器（on late-init 等）：边沿语义，入队一次、错过不补；属性触发器（on property:x=y）：边沿入队+电平匹配——变化的一对按新值判，其余条件按属性区当前值判（第十二章 12.3）。

4. fork 只复制调用线程，多线程进程 fork 后子进程可能持有"不存在的线程"占着的锁→死锁；startZygoteNoThreadCreation（ZygoteInit.java:826）保证 zygote 单线程，fork 出的进程世界状态一致（第十章）。

5. setuid/setgid=1000、setgroups 27 组、capabilities 双位图 13 项、nice-name=system_server、runtime-args、入口类 com.android.server.SystemServer（ZygoteInit.java:711-789，第六章 6.4）。

6. 依赖拓扑排序的显式化：Bootstrap（后序服务的地基：Installer→AMS→PMS）、Core（轻量统计类）、Other（大合唱+phase 广播）；phase 100→1000 是全局进度条，服务在 onBootPhase 挂钩（第七章 7.2/7.3）。

7. 550：系统服务间调用放行（AMS 等在此之后才接受依赖调用）；600：三方应用可启动（PHASE_THIRD_PARTY_APPS_CAN_START）；1000：PHASE_BOOT_COMPLETED（第七/八章）。

8. sys.boot_completed 先置位（AMS.java:5149）再发 BOOT_COMPLETED 广播：属性是全机权威标志（init 据此触发 on property: 段），广播是通知三方 App 的载体；dev.bootcomplete 是设备级（telephony 等）完成标志，在 finishBooting 尾部（第八章 8.2）。

9. 主线：kernel→FirstStageMain（挂分区切根）→re-exec selinux_setup（读策略）→re-exec second_stage（属性区+rc+触发器入队）→late-init 瀑布→post-fs-data（/data 解密）→zygote-start→class_start main→Service::Start（先建 socket :613 再 fork :633）→init.svc.zygote=running（:684）。关键属性：ro.crypto.state、init.svc.zygote（第四/五章）。

12. init 的世界是"顺序的世界"：确定性、公平、无锁需求，单线程是最小复杂度解；system_server 的世界是"并发的世界"：多服务同时响应 Binder，必须线程池。两者不是风格差异，是负载模型差异（第五章）。

13. 先崩的是 AMS：AMS 启动即需查询包信息/装系统包，PMS 未就绪时这些查询失败；而 PMS 依赖的 Installer 与 PlatformCompat 都排在它前面（第七章 7.4 的因果链）。

14. 四个环节都要有接口：生成（设备端 trace 挖掘/云端聚合，产物带指纹与版本签名）、下发（经 Play/OTA 通道、增量更新）、应用（zygote 启动时按 profile 装载子集，替代静态 preloaded-classes）、回退（校验失败回退出厂全量清单）；核心抽象是"预载清单从静态文件变成可验证的动态描述"（第六/九章）。

16. ①孤儿进程处理：App 是 zygote 的子进程，父死子存后要 reparent 并重建 Binder/回调；②预加载世界的生命周期：boot.art/堆与新 system_server 的一致性判定；③跨重启的句柄与 socket 重建（servicemanager 注册表、zygote socket 的所有权）（第六/七/十三章）。

17. 流：property_service 提交共享区→PropertyChanged（init.cpp:333）→门闩已开则 QueuePropertyChange+唤醒主线程→ExecuteOneCommand 遍历 actions 问 CheckEvent→命中者进队列逐命令执行。门闩前写入：不产生事件，但门闩打开时的 QueueAllPropertyActions（空对）按当前值补账——早写的属性不丢，只是延迟命中（第十二章 12.1/12.4）。

18. 直接抛 ZygoteSecurityException（ZygoteConnection.java:192-197）："Client may not specify capabilities"——能力位只能由 native 按 uid 计算（SpecializeCommon :1713）。设计动机：参数解析层只做身份分层检查，权限分配权保留在内核/native 边界，防止调用方自我提权（第十三章 13.1/13.3）。

19. 30 秒（半超时）：WAITED_HALF→只取证不杀（logWatchog doWaitedHalfDump，dropbox tag=pre_watchdog）；60 秒（全超时）：OVERDUE→再取证（tag=watchdog、EventLog WATCHDOG）→询问 IActivityController→无调试器则 killProcess+exit(10)。取证文件：/data/system/dropbox/（第十四章 14.2）。

20. 窗口从 4 分钟放宽到 8 分钟（fatal_crash_window_），8 分钟内死 5 次才 LOG(FATAL)。两道防线独立计时：Watchdog 60s 管"卡死但活着"（杀 system_server→zygote 自毁→算 zygote 重启

一次）；critical 窗口管"反复崩溃"。zygote 默认 window=off（init.zygote64.rc:17），改成 8 后从"无限重启"变为"8 分钟内死 5 次即整机重启"（第十四/十五章）。

术语表（按拼音排序）

术语	所在册	解释
APEX	1/2	可独立升级的模块容器，apexd 在 post-fs-data 挂载
BCB	2	Boot Control Block（misc 分区，bootloader 读它决定启动模式）
boot.img	2	kernel + ramdisk（GKI 后仅 kernel）
boot phase	2	SystemServiceManager 广播的全局进度条（100→1000）
bootchart	2	init 内建启动时间线采集器
COW	2	Copy-On-Write：fork 后共享页，写时才复制
critical	2	rc 服务属性：死则整机重启
dtbo	2	设备树覆盖分区（板级差异）
GKI	2	Generic Kernel Image：boot 只装 Google 签名内核
init.rc	2	init 的配置文件（声明式语言）
Launcher	2	桌面 App（第一个被启动的"应用"）
lazy preload	2	Go 版默认：首 fork 前才加载 preload 清单
preload	2	Zygote 预加载的类/资源/共享库
preloadClasses	2	frameworks/base/preloaded-classes：3000+ 类清单
re-exec	2	进程重新执行自身（init 两阶段的切换机制）
runSelectLoop	2	Zygote 的孵化主循环（poll + fork）
switch_root	2	第一阶段 init 切换根文件系统
system_server	2	系统服务宿主进程（由 zygote fork）
trigger	2	rc 语言的事件触发器（on <trigger>）
USAP	2	UnSpecialized App Process 池（预 fork 备用进程）
vbmeta	2	AVB 的信任根分区

术语	所在册	解释
Zygote	2	App 之母进程（预加载 + COW fork）

排障手册：启动问题定位

先把总纲放在最前面：**任何启动问题都用"锚点回退法"定位**——`sys.boot_completed`、`init.svc.*`、`boot_progress_*`、`processing action` 是四组免费的里程碑，问题出在哪一节，答案通常就在本册对应的章里。

症状关键词	首查命令	对应章
卡 logo 无下文	dmesg 尾部+getprop init.svc	第二/三章
preload 段超时	logcat -b events 过滤 preload	第六章
桌面反复重启	logcat 过滤 WATCHDOG	第十四章
某服务死循环重启	getprop init.svc.*+logcat -s init	第十五章
BOOT_COMPLETED 不来	getprop sys.boot_completed	第八章
整体开机慢	bootchart（第四章 bootchart 实操）	第九章

场景 1：开机卡在 logo，永远进不了动画

症状

text 串口/adb 侧可观测样例（logcat -b kernel 或串口 dmesg）

```
[ 3.901234] init: init second stage started!
[ 4.112345] init: processing action (early-init) ...
[ 5.550000] init: waiting for /dev/.coldboot_done ...
（此后再无 processing action 输出；logcat 无 boot_progress_start）
```

定位命令

bash 逐步收窄（设备可连 adb 时；否则用串口）

```
adb shell dmesg | tail -50 # 内核侧最后一句在做什么
adb shell getprop | grep -E "init.svc|sys.boot" # 哪些服务起来了
adb shell "ls -l /dev/.booting" # 哨兵还在=init 主循环没跑完 late-init
adb shell cat /proc/1/status | grep State # init 是 S(睡眠等事件) 还是 R/D(卡住)
```

根因

- 冷插扫描未完成：`wait_for_coldboot_done`（`init.cpp:960`）等不到 ueventd 写完 `/dev/.coldboot_done`（第二章 2.3.1）——驱动/设备树异常导致 uevent 风暴或漏发；
- 某 rc 命令阻塞：`wait/exec/mount` 卡在不可达的分区或损坏的块设备上（`action.cpp:158-171` 的 50ms 慢命令日志可确认是哪条）；
- 分区挂载失败：first_stage_mount 校验失败后卡在 recovery 分流（第二章 2.1）。

修复

- 根因是冷插：检查内核驱动枚举与 dts 节点，必要时给 ueventd 规则补齐（ueventd.rc）；
- 根因是 rc 阻塞：给 `wait` 加超时、把不可靠的 `exec` 改成 `exec_background`，或把该动作移到 late-fs 之后；
- 根因是挂载：核对 fstab 的 fs_mgr 标志与 AVB 状态（第十一章 GKI/bootconfig 的参数域陷阱）。

深入：第二、三、四章。

场景 2：卡在 Zygote 预加载（boot_progress_preload_start 之后无下文）

症状

text adb logcat -b events 样例

```
boot_progress_start: 1234 ← app_main 起来了
boot_progress_preload_start: 1567 ← 开始 preload
（preload_end 迟迟不出，或出现）
E Zygote: Failed to open /system/etc/preloaded-classes
E art: Abort message: 'Check failed: ...' ← preloadClasses 期间 abort
```

定位命令

bash 三条命令锁定 preload

```
adb logcat -b events | grep -E "boot_progress_(start|preload)" # 时间戳差值=preload 时长
adb logcat -s zygote art # 类加载失败的具体类名
adb shell "head -5 /system/etc/preloaded-classes" # 清单是否可读/被厂商改动
```

根因

- preloaded-classes 清单与 framework 不匹配（OTA 半刷/厂商裁剪），某个类加载即抛异常——`preloadClasses` 逐行 `Class.forName`，一行失败即 fatal（第六章 6.2）；
- /data 未就绪时提前进入 preload：触发器顺序被改（`zygote-start` 跑到了 `post-fs-data` 前，见习题 11 的因果链）；
- WebView/图形驱动预载挂死（`maybePreloadGraphicsDriver` 阻塞在 GPU probe）。

修复

- 对照构建产物校验 preloaded-classes 与 boot.art（`adb shell ls -l /system/framework/*.art`）；
- 恢复 init.rc 触发顺序；临时绕过可对 Go 场景用 `--enable-lazy-preload`（ZygoteInit.java:855）验证"是否 preload 代码路径本身的问题"；
- 图形驱动问题在 vendor 侧 logcat（`logcat -s GraphicsLoader`）。

深入：第六章 6.2/6.3、第十二章 12.4。

场景 3：桌面反复重启（watchdog 杀 system_server）

症状

text adb logcat 样例（关键字按出现顺序）

```
I Watchdog: WAITED_HALF
W Watchdog: *** WATCHDOG KILLING SYSTEM PROCESS: Blocked in monitor on foreground thread
(android.os
.PowerManagerService$...)
W Watchdog: *** GOODBYE!
I Zygote: Exit zygote because system server (pid 812) has terminated
I init: Service 'zygote' (pid 512) received signal 9, killing process group...
（约半分钟后 SystemServer 重新走三阶段，若再次触发则循环）
```

定位命令

bash 从证据到阻塞点

```
adb shell ls -l /data/system/dropbox/ | grep -E "watchdog|system_server" # 事故现场 trace
adb logcat -d | grep -E "WATCHDOG|Blocked in" # 阻塞线程与 subject
adb shell dumpsys dropbox --print watchdog # 完整栈：谁持锁谁在等
adb shell getprop | grep -i watchdog # framework_watchdog.* 计数
```

根因

- Watchdog 判据是**消息队列超时**而非 CPU（第十四章 14.1）：被监控线程（foreground/main/ui 等）上的某个服务 `monitor()` 或消息卡在锁上——通常是等待一个不回话的 native 服务/HAL；
- `System.exit(10)`（`Watchdog.java:848`）后 zygote 自毁（`Zygote.cpp:455-462`）→ 全体 App 被陪葬 → 桌面重启是**结果不是原因**；
- `framework_watchdog.fatal_count` 非零说明已进入崩溃环（`:108-109` 属性、`:843 breakCrashLoop`）。

修复

- 以 dropbox watchdog trace 的持锁链为准修死锁（常是 Binder 同步调用反转：system_server 同步外呼，参见第七章 `Binder.setWarnOnBlocking` 的动机）；
- 临时缓解：`adb shell settings put global watchdog_timeout_millis 120000` 拉长观察窗（`SettingsObserver`，`Watchdog.java:459-497`），仅用于定位；
- 复现期可连调试器抑制击杀（`:829-838` debugger 保护），拿到完整现场。

深入：第十四章全章、第十三章 13.4。

场景 4：某服务被 init 死循环重启，最后整机重启

症状

text adb logcat -s init 样例

```
W init: Service 'mydaemon' (pid 5678) exited with status 1
W init: Service 'mydaemon' (pid 5700) exited with status 1
```

```
...
F init: critical process 'mydaemon' exited 4 times in 4 minutes
（随后 init LOG(FATAL) abort，机器按 target 重启）
```

定位命令

bash 状态机+熔断计数一把抓

```
adb shell getprop init.svc.mydaemon # running/restarting 循环
adb logcat -d -s init # 每次退出的 exit code 与重启间隔
adb shell cat /proc/$(pidof mydaemon)/status 2>/dev/null | grep -E "SigCgt|Uid"
adb logcat -d | grep -E "processing action|Service 'mydaemon'"
```

根因

- 服务自身秒退（配置缺失/SELinux 拒绝/动态库找不到）——每次死亡 Reap 都把 `crash_count_` 加一（`service.cpp:324-350`）；4 分钟窗口内第 5 次触发 `LOG(FATAL)`（`:335`）；
- `onrestart` 挂钩放大：每次重启执行 `onrestart` 命令（`:356`），若挂钩本身失败会加速恶化；
- 二进制依赖的 APEX 未激活即被 class_start（`post-fs-data` 顺序，第十二章 12.4）。

修复

- 先看每次退出的 `status`：1=业务初始化失败（补配置）、信号 6/7=崩溃（抓 tombstone）、126/127=exec 失败（SELinux/路径）；
- 开发期用 `init.svc_debug.no_fatal.<name>=1` 临时关熔断（`:334`），避免机器重启打断取证；
- 上线前用 `host_init_verifier` 校验 rc 与 SELinux 三件套（第四章 4.2）。

深入：第十五章全章、第四章 4.7。

场景 5：BOOT_COMPLETED 迟迟不来

症状

text adb shell getprop 观测样例

```
[sys.boot_completed] [] ← 长时间空
[init.svc.bootanim] [running] ← 动画不退（第八章 8.2 的门闩在等它）
adb logcat -b events 末尾停在 boot_progress_ams_ready 之后
```

定位命令

bash 门闩两端各看一眼

```
adb shell getprop sys.boot_completed; adb shell getprop init.svc.bootanim
adb logcat -d | grep -E "systemReady|bootAnimationComplete|SystemServerTiming"
adb shell dumpsys activity | grep -A3 "mBootAnimationComplete\|mCallFinishBooting"
adb logcat -b events | tail -20 # 最后一个 boot_progress 停在哪
```

根因

- 动画不退：surfaceflinger 未就绪或 `bootanim` 被 vendor 层逻辑卡住（`mBootAnimationComplete` 门闩，`AMS.java:5075`）；

- systemReady 链上某服务未完成：goingCallback 里的服务收尾阻塞（`SystemServer.java:2801-3061`），等 60 秒会转成场景 3；
- 广播已发但接收方拖死：有序广播被某高优先级接收器滞留（`dumpsys activity broadcast` 看 receivers 分发）。

修复

- 动画问题在 SF/bootanim 侧修；临时验证可 `adb shell setprop ctl.stop bootanim` 观察门闩是否放行（注意：这只是旁路验证）；
- 服务收尾阻塞：用 `SystemServerTiming` 日志找最后一个未完成的 traceBegin 段；
- 广播滞留：按 `dumpsys activity broadcasts` 的历史记录定位滞留接收器。

深入：第八章全章、第七章 7.4。

场景 6：整体开机慢，说不出慢在哪

症状：能开机，但冷启动 40 秒+；无崩溃、无熔断，纯粹"慢"。

定位命令

bash 度量三件套（第九章的方法论）

```
adb shell setprop persist.init.bootchart 1 && adb reboot # 采集（第四章实操五步）
adb logcat -d -b events | grep boot_progress # 分段锚点时间戳
adb shell dmesg | grep -E "init: processing action" -A0 # action 粒度时间线
```

根因（按占比概率排序）

- zygote 预加载吃掉 1-2 秒：preloaded-classes 未裁剪（第六章 6.6 的内存-首帧交易）；
- PMS 扫描全机 APK（`SystemServer.java:1233`）：priv-app 数量失控；
- rc 的串行命令链上有 50ms+ 慢命令（`action.cpp:158-171` 逐条警告）；
- 匿名页过多导致 zygote fork 的 COW 基数变大（`ForkCommon` 的 `mallopt(M_PURGE)` 只能省一部分）。

修复：按第九章三经典优化位定向处理（裁剪预载/并行化/USAP），改一项复验一次 bootchart——**没有复验的优化都算没做**。

深入：第九章、第十二章、第六章。